

Bayesian Modeling with RJAGS

Libraries

```
In [2]: options(warn = -1)
library(ggplot2)
library(ggplot2)
library(ggribes)
library(ggextra)
library(dplyr)
library(rjags)
library(coda)
library(openintro)
library(mosaic)
```

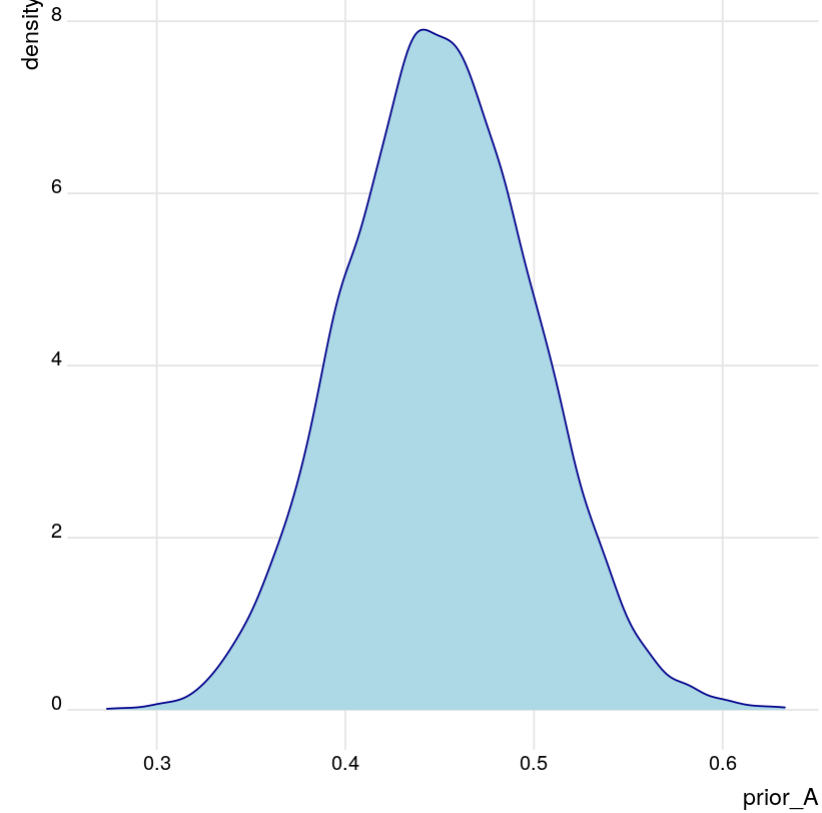
Simulating a Beta prior

Suppose you're running in an election for public office. Let p be your underlying support, the proportion of voters that plan to vote for you. Based on past polls, your prior model of p is captured by a Beta distribution with shape parameters 45 and 55. We will approximate the $p \sim \text{Beta}(45, 55)$ prior using random samples from the `rbeta()` function.

```
In [3]: # Sample 10000 draws from Beta(45,55) prior
prior_A <- rbeta(n = 10000, shape1 = 45, shape2 = 55)

# Store the results in a data frame
prior_sim <- data.frame(prior_A)

# Construct a density plot of the prior sample
ggplot(prior_sim, aes(x = prior_A)) + geom_density(color="darkblue", fill = 'lightblue') + theme_ridges()
```



Comparing & contrasting Beta priors

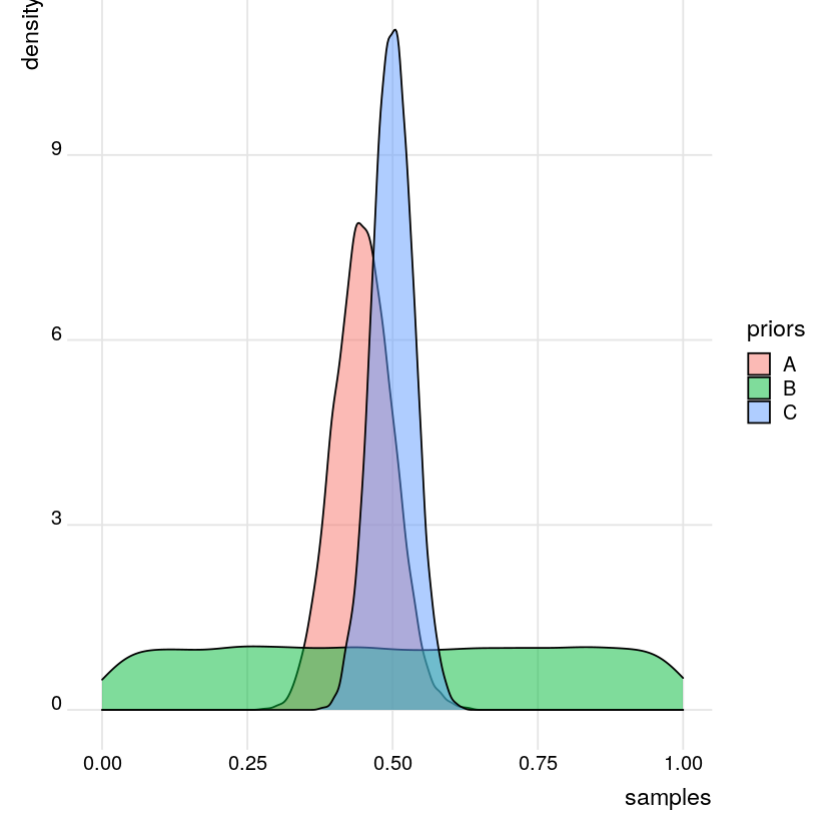
The $\text{Beta}(a, b)$ distribution is defined on the interval from 0 to 1, thus provides a natural and flexible prior for your underlying election support, p . We can tune the Beta shape parameters a and b to produce alternative prior models. We will compare our original $\text{Beta}(45, 55)$ prior with two alternatives: $\text{Beta}(1, 1)$ and $\text{Beta}(100, 100)$.

```
In [4]: # Sample 10000 draws from the Beta(1,1) prior
prior_B <- rbeta(n = 10000, shape1 = 1, shape2 = 1)

# Sample 10000 draws from the Beta(100,100) prior
prior_C <- rbeta(n = 10000, shape1 = 100, shape2 = 100)

# Combine the results in a single data frame
prior_sim <- data.frame(samples = c(prior_A, prior_B, prior_C),
  priors = rep(c("A", "B", "C"), each = 10000))

# Plot the 3 priors
ggplot(prior_sim, aes(x = samples, fill = priors)) + geom_density(alpha = 0.5) + theme_ridges()
```



Prior B reflects vague prior information about p - it gives equal prior weight to all values of p between 0 and 1.

Prior C reflects more prior certainty about p - it has less spread and is centered around a mean that's greater than that for Prior A .

Polling Data

- **parameter**
 - p = proportion of voters that support you
- **data**
 - X = 6 of $n = 10$ or 60% of polled voters plan to vote/support you
- **Insights**
 - You are more likely to have observed these data if $p \approx 0.6$ than if $p < 0.5$.

Modeling the dependence of X on p

- **Poll assumptions:**
- Voters are independent: p = probability that a voter supports you
- X = number of n polled voters that support you (count of successes in n independent trials, each having probability of success p)
- **Conditional distribution of X given p :**

$$X \sim \text{Bin}(n, p)$$

Likelihood

The likelihood function summarizes the likelihood of observing polling data X under different values of the underlying support parameter p . It is a function of p . Thus the likelihood is a function of p that depends on the observed data X , in turn it provides insight into which parameter values are most compatible with the poll.

- high likelihood $\Rightarrow p$ is compatible with the data
- low likelihood $\Rightarrow p$ is not compatible with the data

Simulating the dependence of X on p

In your quest for election to public office, your campaign polls 10 likely voters. Let X be the number that support you. Of course, X varies from sample to sample and depends upon p , your underlying support in the broader population. Since X is a count of successes in 10 independent trials, each having probability of success p , we can model its dependence on p by the Binomial distribution: $\text{Bin}(10, p)$.

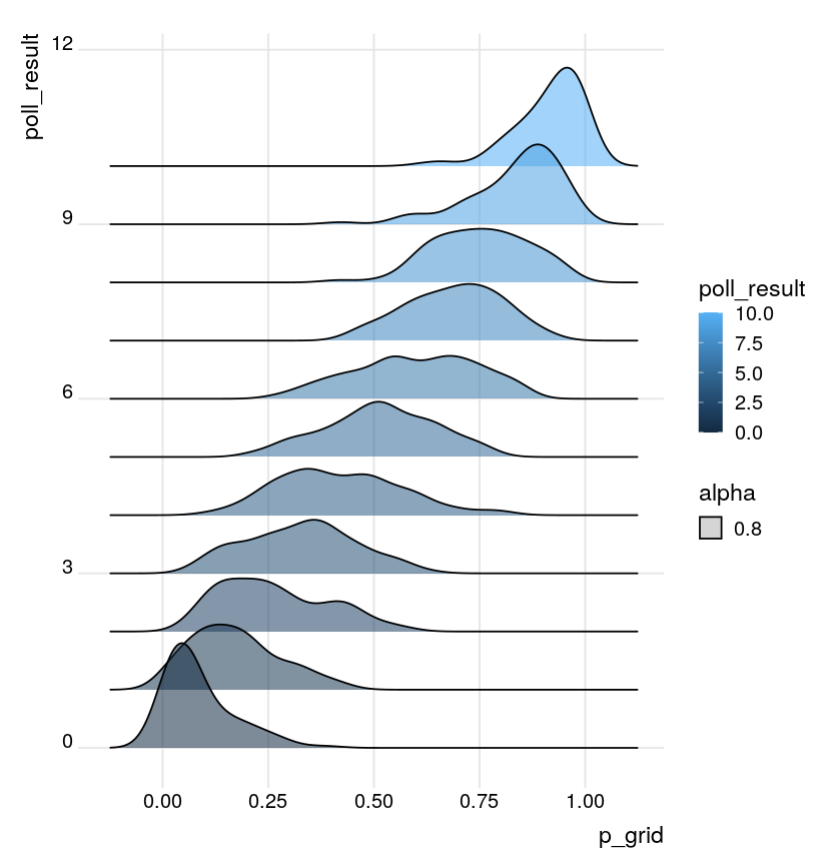
We will simulate the Binomial model using random samples from the `rbinom(n, size, prob)` function. This vectorized function draws n samples from a $\text{Bin}(size, prob)$ distribution. Given a vector of prob values, the first prob value will be used for the first draw, the second prob value will be used for the second draw, etc.

```
In [5]: # Define a vector of 1000 p values from 0 to 1
p_grid <- seq(from = 0, to = 1, length.out = 1000)

# Simulate 1 poll result for each p in p_grid
poll_result <- rbinom(n = 1000, # Number of simulations
  size = 10, # Number of Trials (voters)
  prob = p_grid) # Underlying proportion of success between 0.0 and 1.0

# Create Likelihood_sim data frame
likelihood_sim <- data.frame(p_grid, poll_result)

# Density plots of p_grid grouped by poll_result
ggplot(likelihood_sim, aes(x = p_grid, y = poll_result, group = poll_result, fill = poll_result, alpha = 0.8)) +
  geom_density_ridges() + theme_ridges()
```



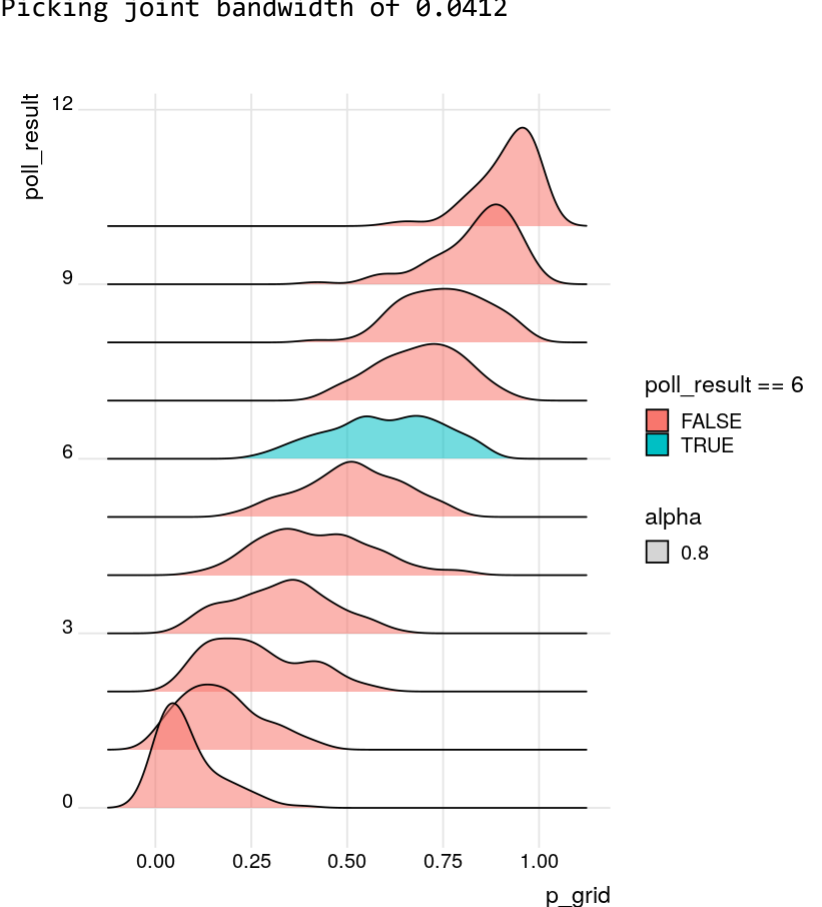
Notice that polls in which 0 people supported you (`poll_result == 0`) correspond to smaller values of underlying support p (`p_grid`). The opposite is true for polls in which all 10 people supported you.

Approximating the likelihood function

The first election poll is in! $X = 6$ of 10 polled voters plan to vote for you. You can use these data to build insight into your underlying support p . To this end, we will use the `likelihood_sim` data frame. This contains the values of X (`poll_result`) simulated from each of 1,000 possible values of p between 0 to 1 (`p_grid`).

Here we supply a fill condition in order to highlight the distribution which corresponds to our observed `poll_result`, $X = 6$. This provides insight into which values of p are the most compatible with our observed poll data.

```
In [6]: # Density plots of p_grid grouped by poll_result
ggplot(likelihood_sim, aes(x = p_grid, y = poll_result, group = poll_result, fill = poll_result == 6, alpha = 0.8)) +
  geom_density_ridges() + theme_ridges()
```



This is a scaled approximation of the likelihood function. It indicates that the simulated surveys in which 6 of 10 voters supported you corresponded to underlying support p that ranged from approximately 0.25 to 1, with p around 0.6 being the most common.

Posterior model of p

- **Prior:** $p \sim \text{Beta}(45, 55)$
- **likelihood:** $X \sim \text{Bin}(10, p)$
- **Bayes' Rule:** posterior $\propto$ prior $\times$ likelihood

Getting Started with RJAGS

Bayesian Models in RJAGS: DEFINE

- $X \sim \text{Bin}(n, p)$
- $p \sim \text{Beta}(a, b)$

Define, compile, and simulate

In your election quest, let p be the proportion of the underlying voting population that supports you. Built from previous polls & election data, your prior model of p is a $\text{Beta}(a, b)$ with shape parameters $a = 45$ and $b = 55$. For added insight into p , you also polled n potential voters. The dependence of X , the number of these voters that support you, on p is modeled by the $\text{Bin}(n, p)$ distribution.

In the completed poll, $X = 6$ of $n = 10$ voters supported you. The next goal is to update the model of p in light of these observed polling data. To this end, we will use the `rjags` package to approximate the posterior model of p . We break this down into the 3 `rjags` steps: *define, compile, simulate*.

```
In [7]: # DEFINE the model
vote_model <- "model{
  # Likelihood model for X
  X ~ dbin(p, n)

  # Prior model for p
  p ~ dbeta(a, b)
}"

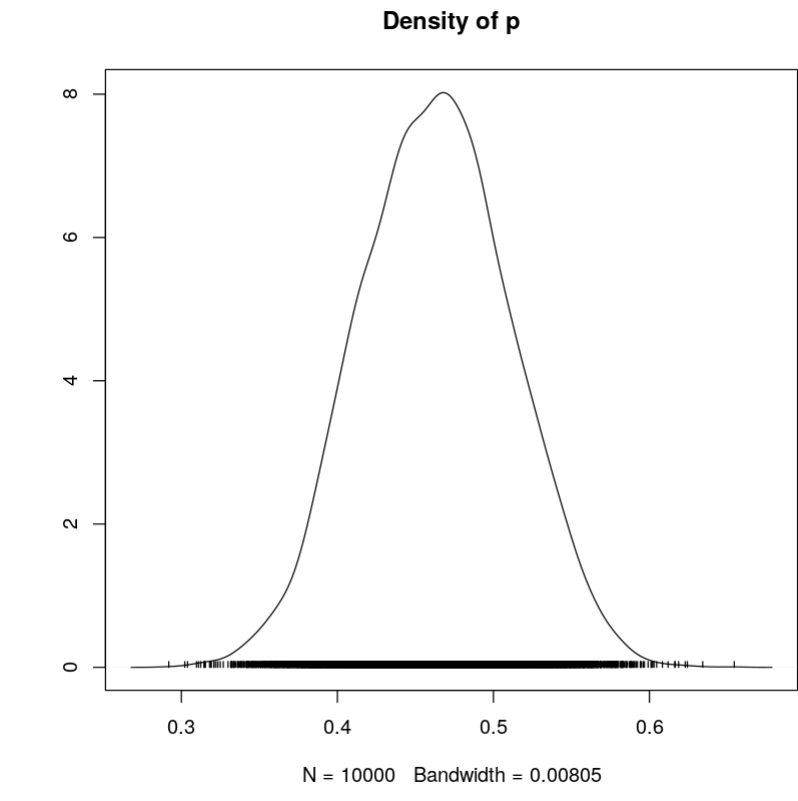
# COMPILE the model
vote_jags <- jags.model(textConnection(vote_model),
  data = list(a = 45, b = 55, X = 6, n = 10),
  inits = list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 100))

# SIMULATE the posterior
vote_sim <- coda.samples(model = vote_jags, variable.names = c("p"), n.iter = 10000)

# PLOT the posterior
plot(vote_sim, trace = FALSE)

Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 1
Unobserved stochastic nodes: 1
Total graph size: 5

Initializing model
```



Notice that after observing a poll in which 6 of 10 (60%) of voters supported you, your updated posterior optimism and certainty about your underlying support, p , are slightly higher than they were prior to the poll.

Updating the posterior

The posterior model of your underlying election support p is informed by both the prior model of p and polling data X . The code below reminds us of the posterior that evolved from the original prior ($\text{Beta}(45, 55)$) and original poll data ($X = 6$ of $n = 10$ polled voters support you).

In a 3-steps we will explore how using a different prior model or observing new data (or a combination of the two) might impact the posterior.

($X = 6, n = 10$)

Here we re-compile, simulate, and plot the p posterior to reflect the setting in which we start with a $\text{Beta}(1, 1)$ prior but observe the same polling data ($X = 6, n = 10$). NOTE: Recall that $\text{Beta}(1, 1)$ is uniform across the $(0, 1)$ interval.

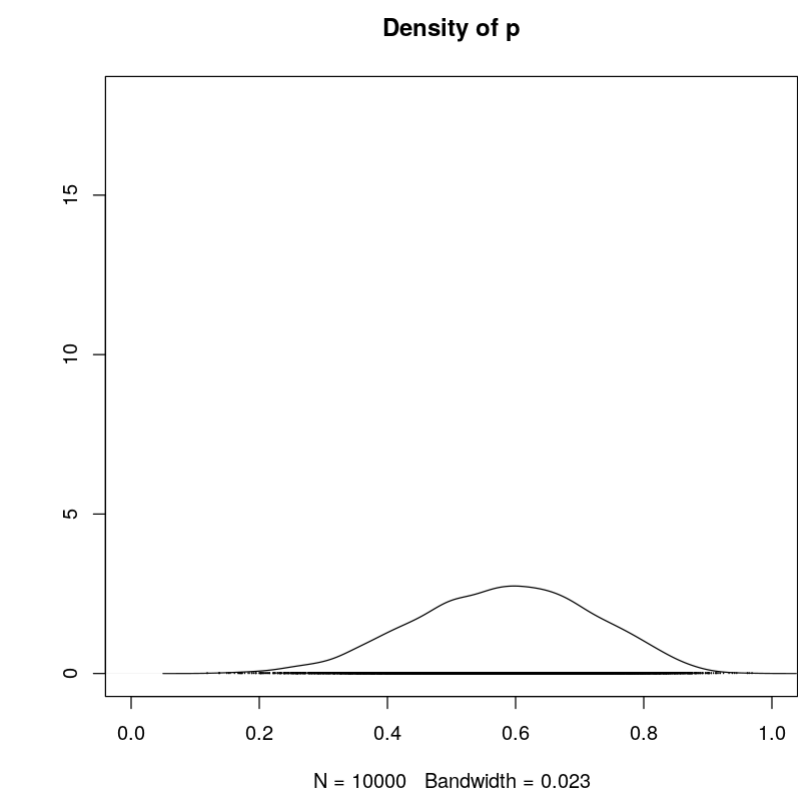
```
In [8]: # COMPILE the model
vote_jags <- jags.model(textConnection(vote_model),
  data = list(a = 1, b = 1, X = 6, n = 10),
  inits = list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 100))

# SIMULATE the posterior
vote_sim <- coda.samples(model = vote_jags, variable.names = c("p"), n.iter = 10000)

# PLOT the posterior
plot(vote_sim, trace = FALSE, xlim = c(0,1), ylim = c(0,18))

Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 1
Unobserved stochastic nodes: 1
Total graph size: 5

Initializing model
```



($X = 220, n = 400$)

In a new poll, 214 of 390 voters support you. Combined with the first poll, a total $X = 220$ of $n = 400$ voters support you. We Re-compile, simulate, and plot the p posterior to reflect these combined poll results and a $\text{Beta}(1, 1)$ prior.

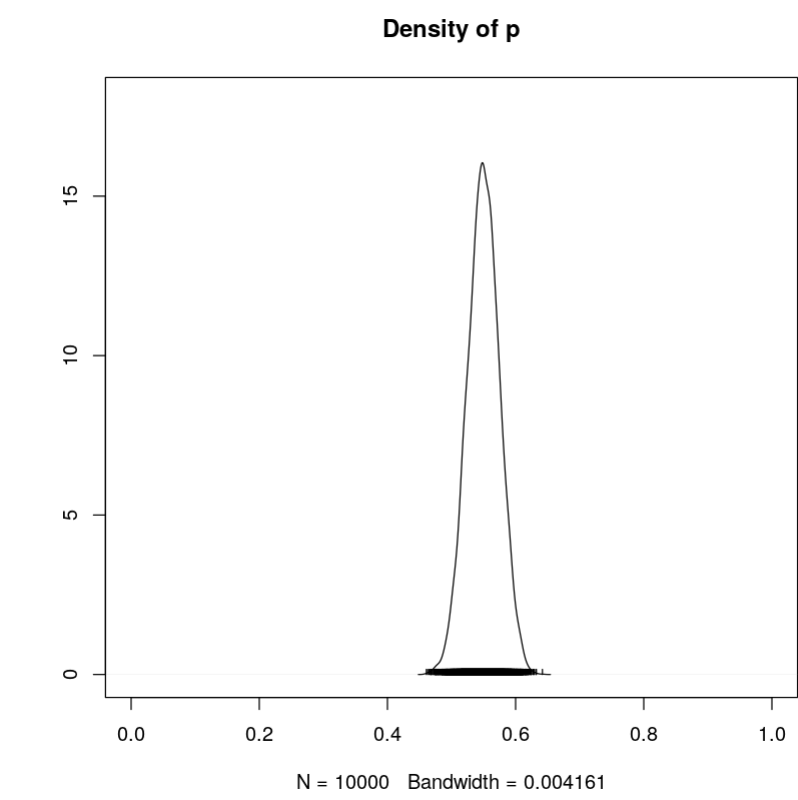
```
In [9]: # COMPILE the model
vote_jags <- jags.model(textConnection(vote_model),
  data = list(a = 1, b = 1, X = 220, n = 400),
  inits = list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 100))

# SIMULATE the posterior
vote_sim <- coda.samples(model = vote_jags, variable.names = c("p"), n.iter = 10000)

# PLOT the posterior
plot(vote_sim, trace = FALSE, xlim = c(0,1), ylim = c(0,18))

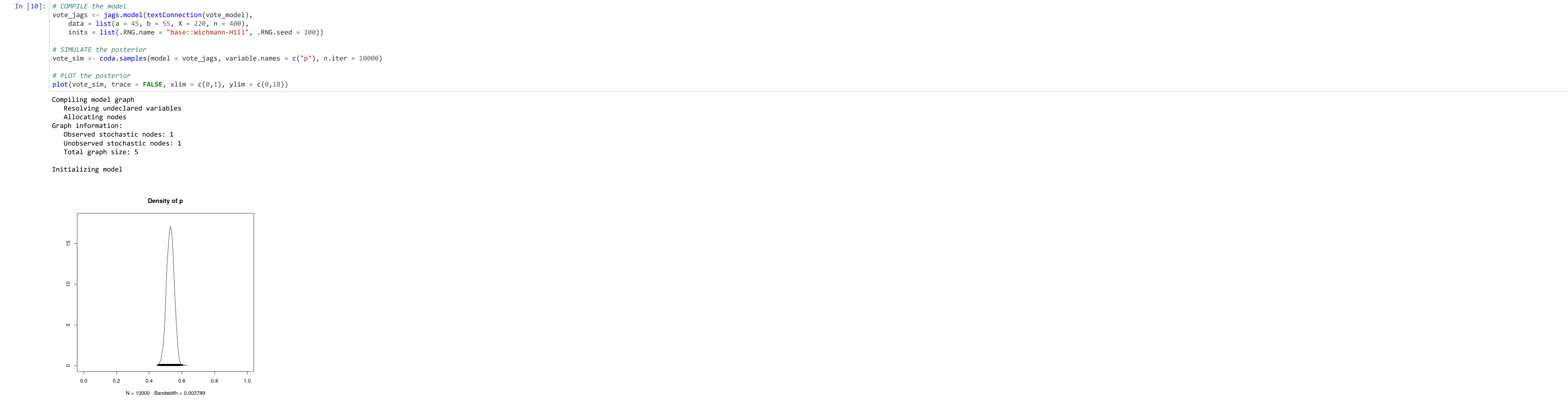
Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 1
Unobserved stochastic nodes: 1
Total graph size: 5

Initializing model
```



($\text{Beta}(45, 55)$) prior

Finally we, re-compile, simulate, and plot the p posterior to reflect the combined poll results ($X = 220$ of $n = 400$) and our original $\text{Beta}(45, 55)$ prior.



- Even in light of the same data, different priors lead to different posteriors.
- The influence of the prior on the posterior diminishes as the sample size of your data increases.
- As sample size increases, your posterior understanding becomes more precise.

The Normal-Normal Model

Sleep deprivation

Research Question:

- How does sleep deprivation impact reaction time?

The Study

- measure reaction time on Day 0
- restrict sleep to 3 hours per night
- measure reaction time on Day 3
- measure the change in reaction time

Modeling change in reaction time

Y_i = change in reaction time (ms)

Assume

- Y_i are Normally distributed around some average change in reaction time m with standard deviation s .

$$Y_i \sim N(m, s^2)$$

Prior model for parameter m :

- Y_i = change in reaction time (ms)
- $Y_i \sim N(m, s^2)$
- m = average Y_i

Prior information:

- With normal sleep, average reaction time is ~ 250 ms
- Expect average to $\nearrow$ by ~ 50 ms
- Average is unlikely to $\searrow$ & unlikely to $\nearrow$ by more than ~ 150 ms

Prior model for parameter s :

- Y_i = change in reaction time (ms)
- $Y_i \sim N(m, s^2)$
- s = standard deviation of Y_i

Prior information:

- $s > 0$
- With normal sleep, s.d. in reaction times is ~ 30 ms
- s is equally likely to be anywhere from 0 to 200 ms

The Normal-Normal Model

Likelihood:

- $Y_i \sim N(m, s^2)$

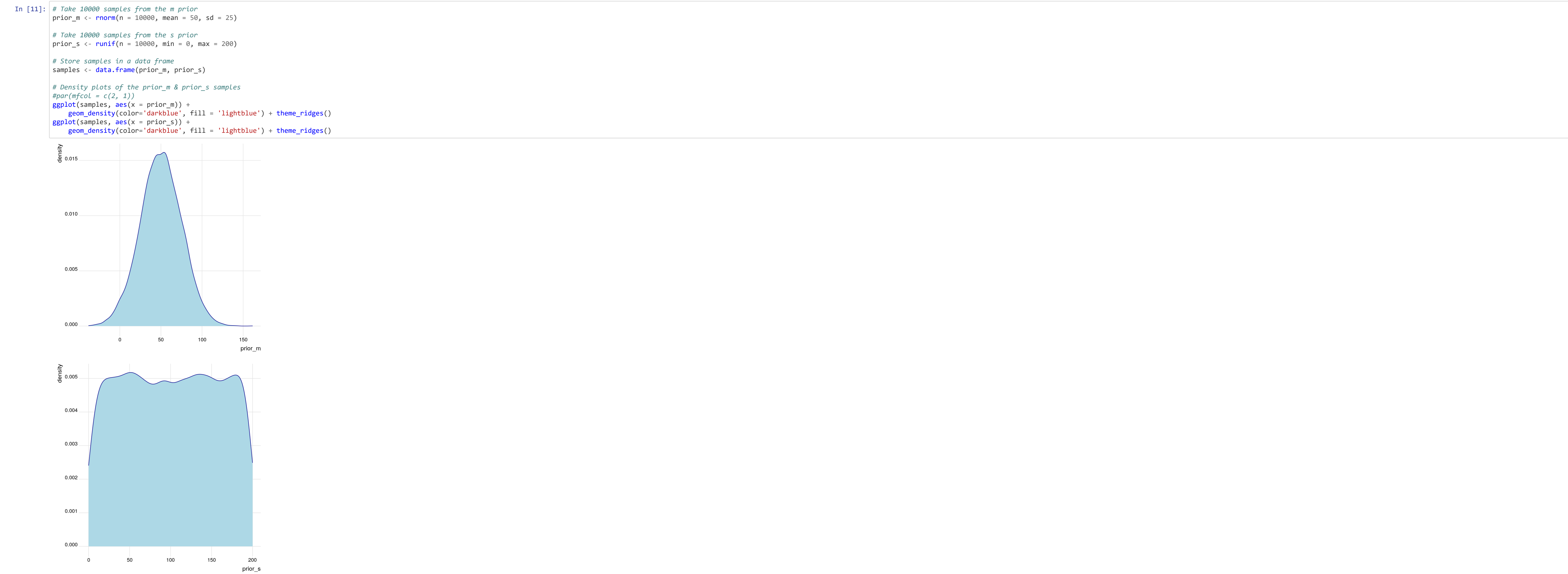
Priors:

- $m \sim N(50, 25^2)$
- $s \sim Unif(0, 200)$

Normal-Normal priors

Researchers developed a test to evaluate the impact of sleep deprivation on reaction time. For subject i , let Y_i be the change in reaction time (in ms) after 3 sleep deprived nights. Of course, people react differently to sleep deprivation. It's reasonable to assume that Y_i are Normally distributed around some average m with standard deviation s : $Y_i \sim N(m, s^2)$.

In the first step of our Bayesian analysis, we'll simulate the following prior models for parameters m and s : $m \sim N(50, 25^2)$ and $s \sim Unif(0, 200)$.



Sleep study data

Researchers enrolled 18 subjects in a sleep deprivation study. Their observed sleep study data are loaded in the DataFrame `sleep_study`. These data contain the `day_0` reaction times and `day_3` reaction times after 3 sleep deprived nights for each subject.

We will define and explore `diff_3`, the observed difference in reaction times for each subject.


```
In [12]: # Check out the first 6 rows of sleep_study
sleep_study <- read.csv('https://assets.datacamp.com/production/repositories/2096/datasets/62737a3d23519485d7bfe3eceb85be0f97a87862/sleep_study.csv',
                        header = TRUE)
head(sleep_study)

# Define diff_3
sleep_study <- sleep_study %>%
  mutate(diff_3 = day_3 - day_0)

# Histogram of diff_3
ggplot(sleep_study, aes(x = diff_3)) +
  geom_histogram(binwidth = 20, color='darkblue', fill = 'lightblue') + theme_ridges()

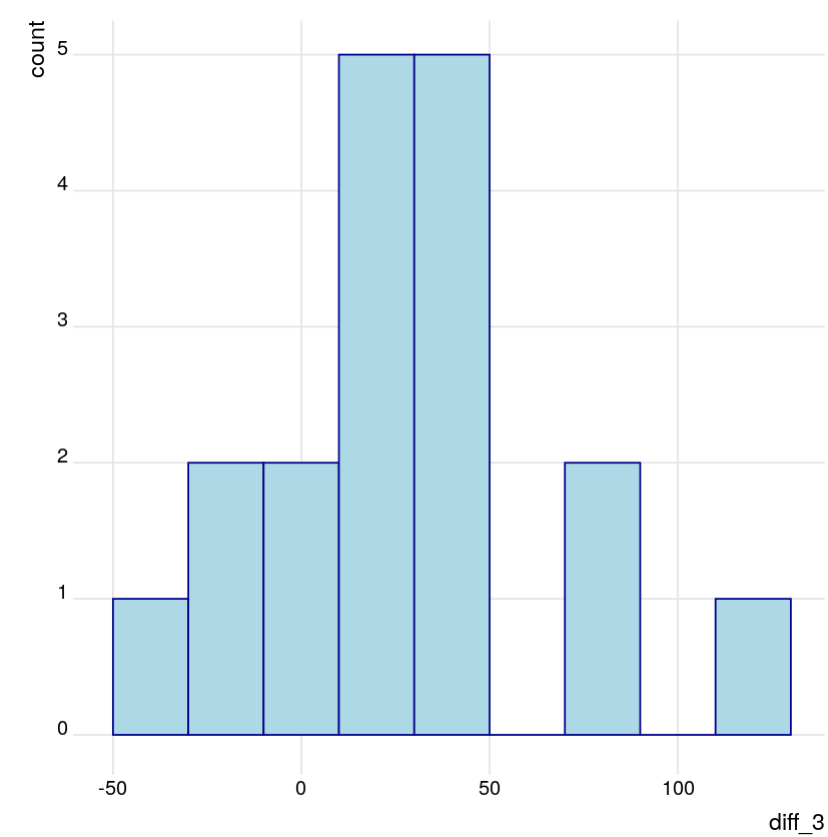
# Mean and standard deviation of diff_3
sleep_study %>%
  summarize(mean(diff_3), sd(diff_3))
```

A data frame: 6 × 3

	subject	day_0	day_3
	<int>	<dbl>	<dbl>
1	308	249.5600	321.4398
2	309	222.7339	204.7070
3	310	199.0539	232.8416
4	330	321.5426	285.1330
5	331	287.6079	320.1153
6	332	234.8606	309.7688

A data frame: 1 × 2

mean(diff_3)	sd(diff_3)
<dbl>	<dbl>
26.34021	37.20764



Reaction times increased by an average of ~ 26 ms with a standard deviation of ~ 37 ms. Further, only 4 of the 18 test subjects had faster reaction times on day 3 than on day 0.

DEFINE the Normal-Normal for RJAGS

- $Y_i \sim N(m, s^2)$ for i in $1, 2, \dots, 18$
- NOTE:** precision = variance⁻¹ = s^{-2}
- $m \sim N(50, 25^2)$
- $s \sim \text{Unif}(0, 200)$

Define, compile, & simulate the Normal-Normal

Upon observing the change in reaction time Y_i for each of the 18 subjects i enrolled in the sleep study, WE can update our posterior model of the effect of sleep deprivation on reaction time. This requires the combination of insight from the *likelihood* and *prior* models:

- likelihood:** $Y_i \sim N(m, s^2)$
- Priors:** $m \sim N(50, 25^2)$ and $s \sim \text{Unif}(0, 200)$

```
In [13]: # DEFINE the model
sleep_model <- "model{
  # Likelihood model for Y[i]
  for(i in 1:length(Y)) {
    Y[i] ~ dnorm(m, s^(-2))
  }

  # Prior models for m and s
  m ~ dnorm(50, 25^(-2))
  s ~ dunif(0, 200)
}"

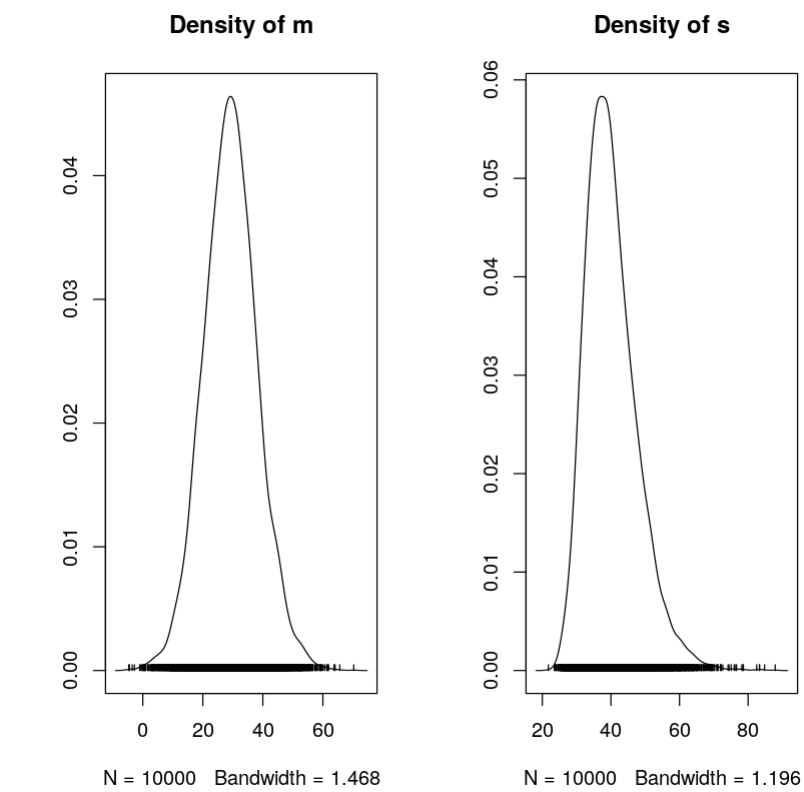
# COMPILE the model
sleep_jags <- jags.model(
  textConnection(sleep_model),
  data = list(Y = sleep_study$diff_3),
  inits = list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 1989)
)

# SIMULATE the posterior
sleep_sim <- coda.samples(model = sleep_jags, variable.names = c("m", "s"), n.iter = 10000)

# PLOT the posterior
plot(sleep_sim, trace = FALSE)
```

Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 18
Unobserved stochastic nodes: 2
Total graph size: 28

Initializing model



We're more certain that average reaction time increases but that the magnitude of the increase is less than assumed prior to observing the data.

Markov chains

Markov chains are stochastic models that describe sequences of possible events. Each event comes from a set of outcomes, and each outcome determines which outcome occurs next, according to a fixed set of probabilities. An important feature of Markov chains is that they are **memoryless**: everything that you would possibly need to predict the next event is available in the current state, and no new information comes from knowing historical events.

- RJAGS goal:** Utilize Markov chains to approximate posteriors that are otherwise too complicated to define or sample
- The m Markov chain traverses the sample space of m , mimics a random sample, and converges to the posterior.

Storing Markov chains

Let m be the average change in reaction time after 3 days of sleep deprivation. Previously we obtained an approximated a sample of 10,000 draws from the posterior model of m . We stored the resulting mcmc.list object as `sleep_sim`. The sample of m values in `sleep_sim` is a dependent **Markov chain**, the distribution of which converges to the posterior.

```
In [14]: # Check out the head of sleep_sim
head(sleep_sim)

# Store the chains in a data frame
sleep_chains <- data.frame(sleep_sim[[1]], iter = 1:10000)

# Check out the head of sleep_chains
head(sleep_chains)

[[1]]
Markov Chain Monte Carlo (MCMC) output:
Start = 1801
End = 1807
Thinning interval = 1
      m      s
[1,] 17.25796 31.46256
[2,] 34.58469 37.88655
[3,] 36.45480 39.58056
[4,] 25.00971 39.69494
[5,] 29.95475 35.98001
[6,] 28.43894 37.46466
[7,] 38.32427 35.44861

attr(,"class")
[1] "mcmc.list"
```

A data frame: 6 × 3

	m	s	iter
	<dbl>	<dbl>	<int>
1	17.25796	31.46256	1
2	34.58469	37.88655	2
3	36.45480	39.58056	3
4	25.00971	39.69494	4
5	29.95475	35.98001	5
6	28.43894	37.46466	6

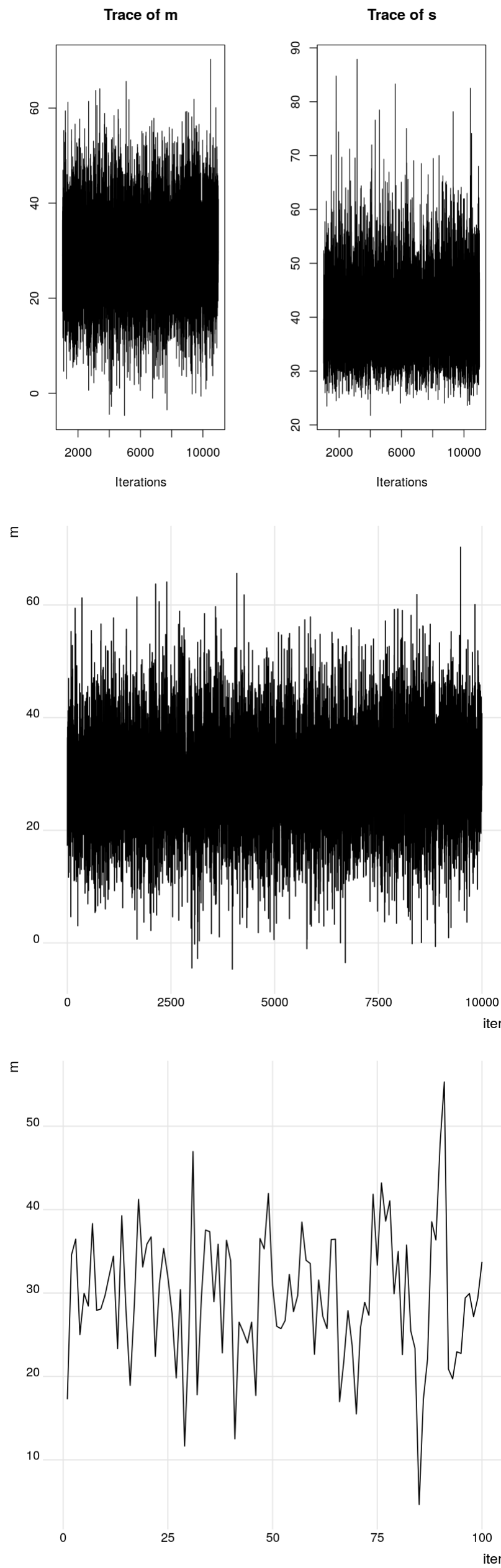
Markov chain trace plots

A trace plot provides a visualization of a Markov chain's longitudinal behavior. Specifically, a trace plot for the m chain plots the observed chain value (y-axis) against the corresponding iteration number (x-axis). We will construct trace plots of the m chain.


```
In [15]: # Use plot() to construct trace plots of the m and s chains
plot(sleep_sim, density = FALSE)

# Use ggplot() to construct a trace plot of the m chain
ggplot(sleep_chains, aes(x = iter, y = m)) +
  geom_line() + theme_ridges()

# Trace plot the first 100 iterations of the m chain
ggplot(sleep_chains[1:100, ], aes(x = iter, y = m)) +
  geom_line() + theme_ridges()
```



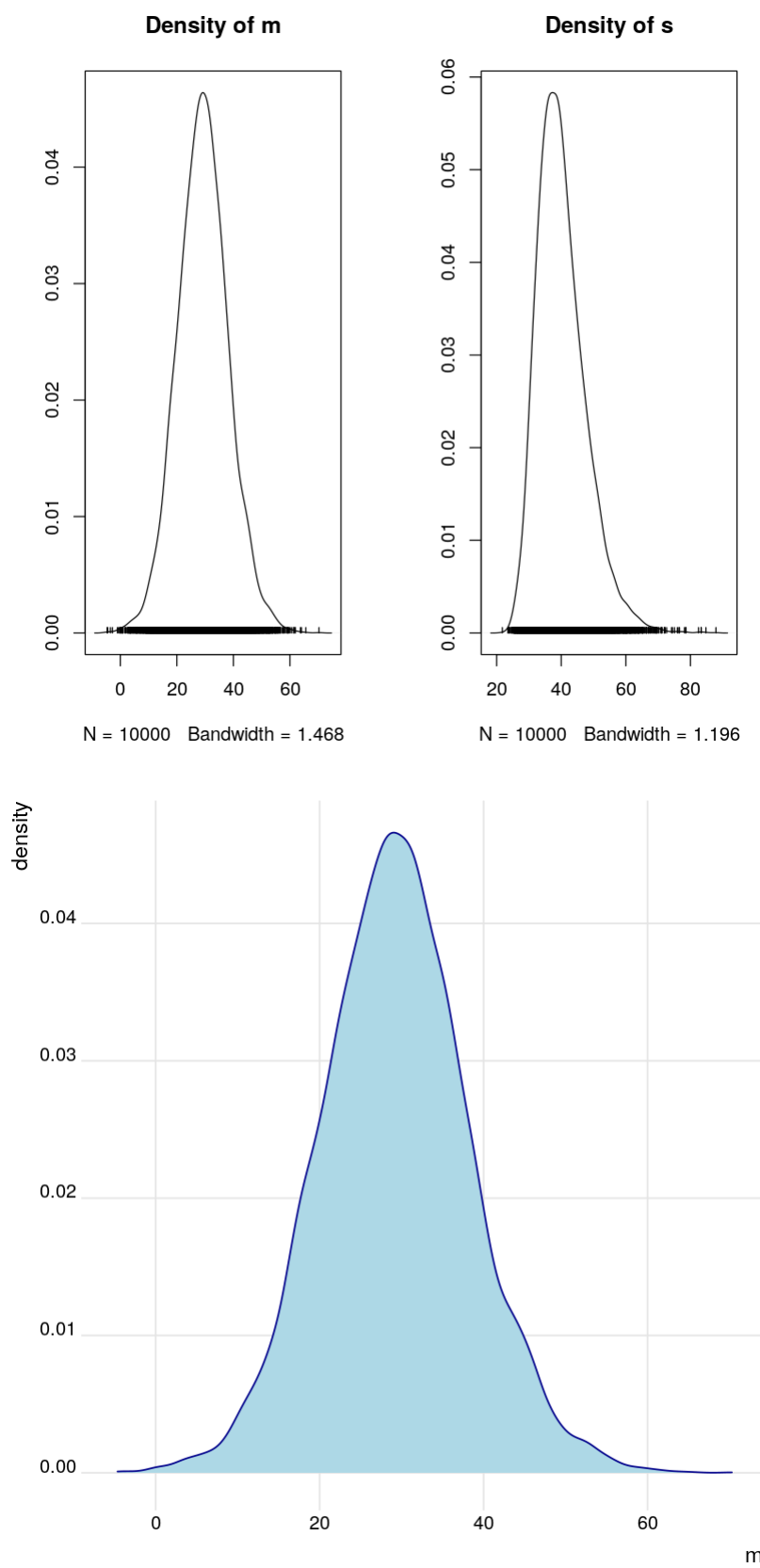
Note that the longitudinal behavior of the chain appears quite random and that the trend remains relatively constant. This is a good thing - it indicates that the Markov chain (likely) converges quickly to the posterior distribution of m .

Markov chain density plots

Whereas a trace plot captures a Markov chain's longitudinal behavior, a density plot illustrates the final distribution of the chain values. In turn, the density plot provides an approximation of the posterior model. We will construct and examine density plots of the m Markov chain.

```
In [16]: # Use plot() to construct density plots of the m and s chains
plot(sleep_sim, trace = FALSE)

# Use ggplot() to construct a density plot of the m chain
ggplot(sleep_chains, aes(x = m)) +
  geom_density(color='darkblue', fill = 'lightblue') + theme_ridges()
```



These density plots approximate the posterior models of m and s .

Markov chain diagnostics & reproducibility

A Good Markov Chain:

- Stable and constant variance
- Stable and constant trend
- Correct by increasing number of iterations

Reproducibility:

- Set the RNG parameters **name** and **seed** for model reproducibility

Examine the following diagnostics:

- trace plots
- multiple chain output
- standard errors: (naïve) standard error

$$\sigma \div \sqrt{\text{number of iterations}}$$

Multiple chains

Trace plots help us diagnose the quality of a Markov chain simulation. A "good" Markov chain will exhibit stability as the chain length increases and consistency across repeated simulations, or multiple chains. We will use **RJAGS** to run and construct trace plots for four parallel chains.


```
In [17]: # COMPiLE the model
sleep_jags_multi <- jags.model(textConnection(sleep_model), data = list(Y = sleep_study$diff_3), n.chains = 4)

# SIMULATE the posterior
sleep_sim_multi <- coda.samples(model = sleep_jags_multi, variable.names = c("m", "s"), n.iter = 1000)

# Check out the head of sleep_sim_multi
head(sleep_sim_multi)

# Construct trace plots of the m and s chains
plot(sleep_sim_multi, density = FALSE)

Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 18
Unobserved stochastic nodes: 2
Total graph size: 28

Initializing model

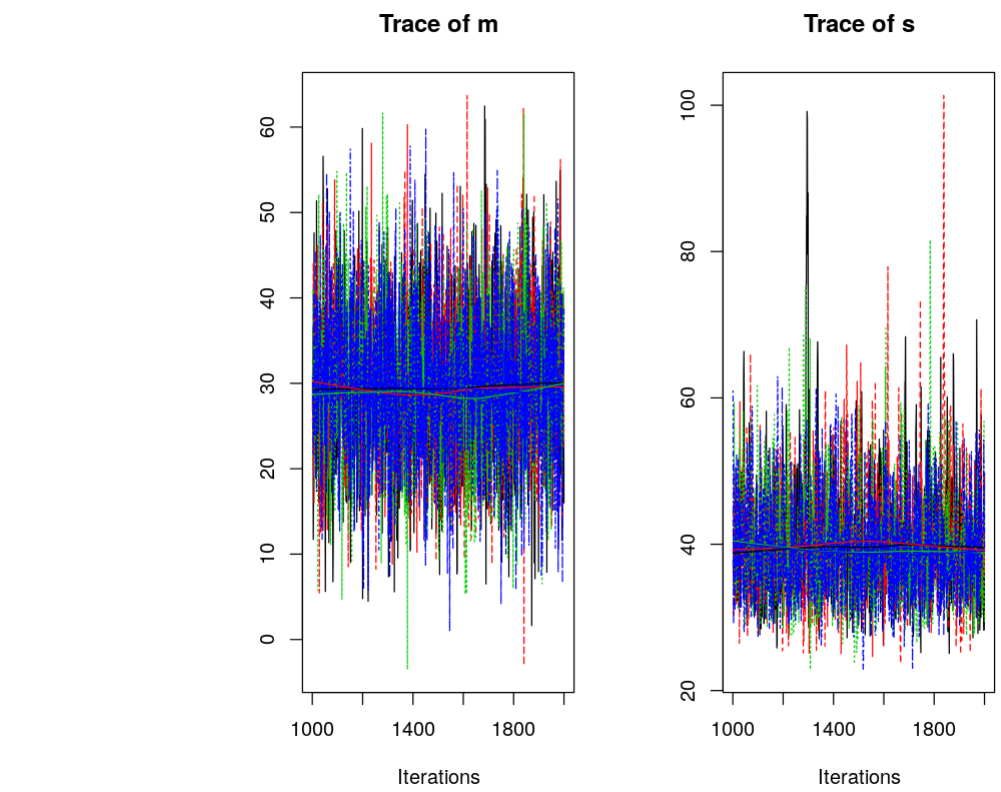
[[1]]
Markov Chain Monte Carlo (MCMC) output:
Start = 1001
End = 1007
Thinning interval = 1
m s
[1,] 26.95139 37.84838
[2,] 36.64104 36.26695
[3,] 24.65847 35.90455
[4,] 11.71905 42.25510
[5,] 18.31079 34.33902
[6,] 32.46486 36.35246
[7,] 47.61596 45.58289

[[2]]
Markov Chain Monte Carlo (MCMC) output:
Start = 1001
End = 1007
Thinning interval = 1
m s
[1,] 25.45950 44.78539
[2,] 31.44026 30.26747
[3,] 28.71086 30.02559
[4,] 44.18226 39.92576
[5,] 19.59408 39.10502
[6,] 22.76513 48.27364
[7,] 16.02904 42.43653

[[3]]
Markov Chain Monte Carlo (MCMC) output:
Start = 1001
End = 1007
Thinning interval = 1
m s
[1,] 25.03641 37.39892
[2,] 26.60073 37.03494
[3,] 41.09780 40.73958
[4,] 28.80065 40.52714
[5,] 27.98351 39.43210
[6,] 35.94804 31.57855
[7,] 29.31679 38.40879

[[4]]
Markov Chain Monte Carlo (MCMC) output:
Start = 1001
End = 1007
Thinning interval = 1
m s
[1,] 30.60809 50.92386
[2,] 28.51053 36.47639
[3,] 21.78438 29.11053
[4,] 27.70521 50.98422
[5,] 24.45239 46.14919
[6,] 40.56091 36.64938
[7,] 33.67518 34.38174

attr(,"class")
[1] "mcmc.list"
```



The most important thing to notice here is the similarity and stability among the 4 parallel chains. This provides some reassurance about the quality and consistency of our Markov chain simulation.

Naive standard errors

The mean of the m Markov chain provides an estimate of the posterior mean of m . The naive standard error provides a measure of the potential error in this estimate. In turn, we can use this measure to determine an appropriate chain length. For example, suppose your goal is to estimate the posterior mean of m within a standard error of 0.1 ms. If your observed naive standard error exceeds this target, no problem! Simply run a longer chain - the error in using a Markov chain to approximate a posterior tends to decrease as chain length increases.

```
In [18]: # SIMULATE the posterior
sleep_sim_1 <- coda.samples(model = sleep_jags, variable.names = c("m", "s"), n.iter = 1000)

# Summarize the m and s chains of sleep_sim_1
summary(sleep_sim_1)

# RE-SIMULATE the posterior
sleep_sim_2 <- coda.samples(model = sleep_jags, variable.names = c("m", "s"), n.iter = 10000)

# Summarize the m and s chains of sleep_sim_2
summary(sleep_sim_2)

Iterations = 11001:12000
Thinning interval = 1
Number of chains = 1
Sample size per chain = 1000

1. Empirical mean and standard deviation for each variable,
plus standard error of the mean:

      Mean      SD Naive SE Time-series SE
m 29.55 9.032   0.2856      0.2856
s 40.81 7.654   0.2420      0.3414

2. Quantiles for each variable:

      2.5%   25%   50%   75%  97.5%
m 12.00 23.62 29.31 35.58 47.48
s 28.34 35.25 39.97 45.41 58.60

Iterations = 12001:22000
Thinning interval = 1
Number of chains = 1
Sample size per chain = 10000

1. Empirical mean and standard deviation for each variable,
plus standard error of the mean:

      Mean      SD Naive SE Time-series SE
m 29.33  8.980   0.08080      0.0942
s 40.12 7.483   0.07483      0.1115

2. Quantiles for each variable:

      2.5%   25%   50%   75%  97.5%
m 11.69 23.25 29.25 35.24 47.55
s 28.61 34.89 39.07 44.25 57.62
```

Reproducibility

Now that we've completed some Markov chain diagnostics, we're ready to finalize our **RJAGS** simulation. To this end, reproducibility is crucial. To obtain reproducible simulation output, we must set the **seed** of the **RJAGS** random number generator. This works differently than in base R.

```
In [19]: # COMPiLE the model
sleep_jags <- jags.model(textConnection(sleep_model), data = list(Y = sleep_study$diff_3),
  inits = list(.RNG.name = "base::Mchmann-Hill", .RNG.seed = 1989))

# SIMULATE the posterior
sleep_sim <- coda.samples(model = sleep_jags, variable.names = c("m", "s"), n.iter = 10000)

# Summarize the m and s chains of sleep_sim
summary(sleep_sim)

Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 18
Unobserved stochastic nodes: 2
Total graph size: 28

Initializing model

Iterations = 1001:11000
Thinning interval = 1
Number of chains = 1
Sample size per chain = 10000

1. Empirical mean and standard deviation for each variable,
plus standard error of the mean:

      Mean      SD Naive SE Time-series SE
m 29.29  8.980   0.08080      0.08854
s 40.19 7.519   0.07519      0.11557

2. Quantiles for each variable:

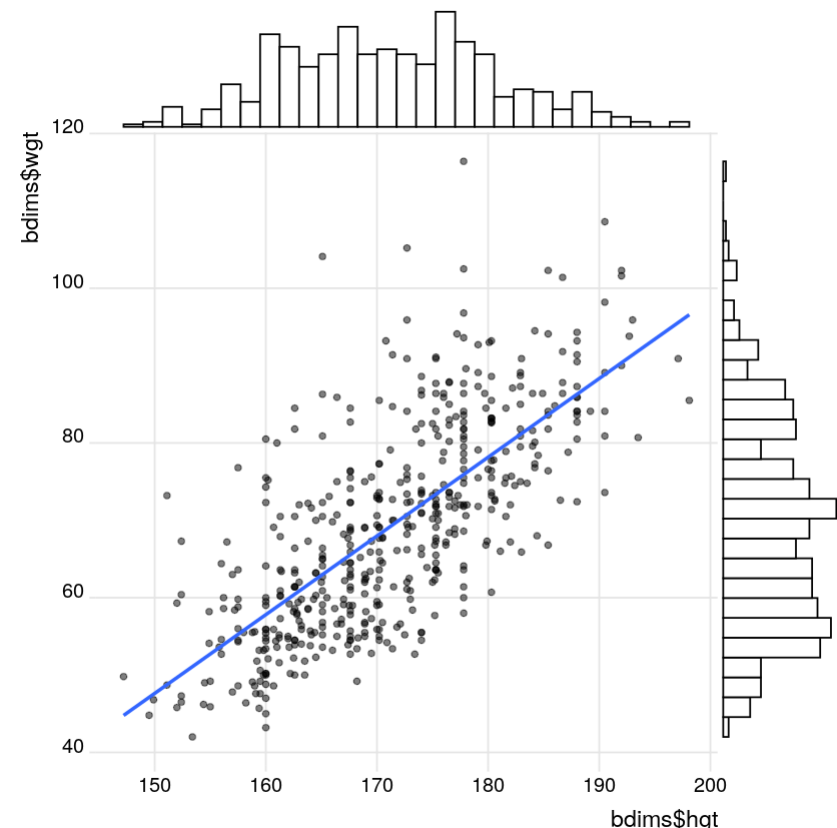
      2.5%   25%   50%   75%  97.5%
m 11.70 23.34 29.16 35.05 47.45
s 28.68 34.85 39.12 44.39 57.79
```

A simple Bayesian regression model

```
In [20]: data(bdms)

p <- ggplot(bdms, aes(bdms$htgt, bdms$wgt)) +
  geom_point(alpha=0.5) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_ridges()
ggMarginal(p, type = "histogram", fill='white')

'geom_smooth()' using formula 'y ~ x'
'geom_smooth()' using formula 'y ~ x'
```



Modeling weight by height

- Y_i = weight of adult i (kg)
- X_i = height of adult i (cm)

Model:

- $Y_i \sim N(m_i, s^2)$
- $m_i = a + bX_i$

Bayesian regression model

- $Y_i \sim N(m_i, s^2)$
- $m_i = a + bX_i$
- $a = y - \text{intercept}$
 - Value of m_i when $X_i = 0$
- $b = \text{slope}$
 - Rate of change in weight (kg) per 1 cm increase in height
- $s = \text{residual standard deviation}$
 - Individual deviation from trend m_i

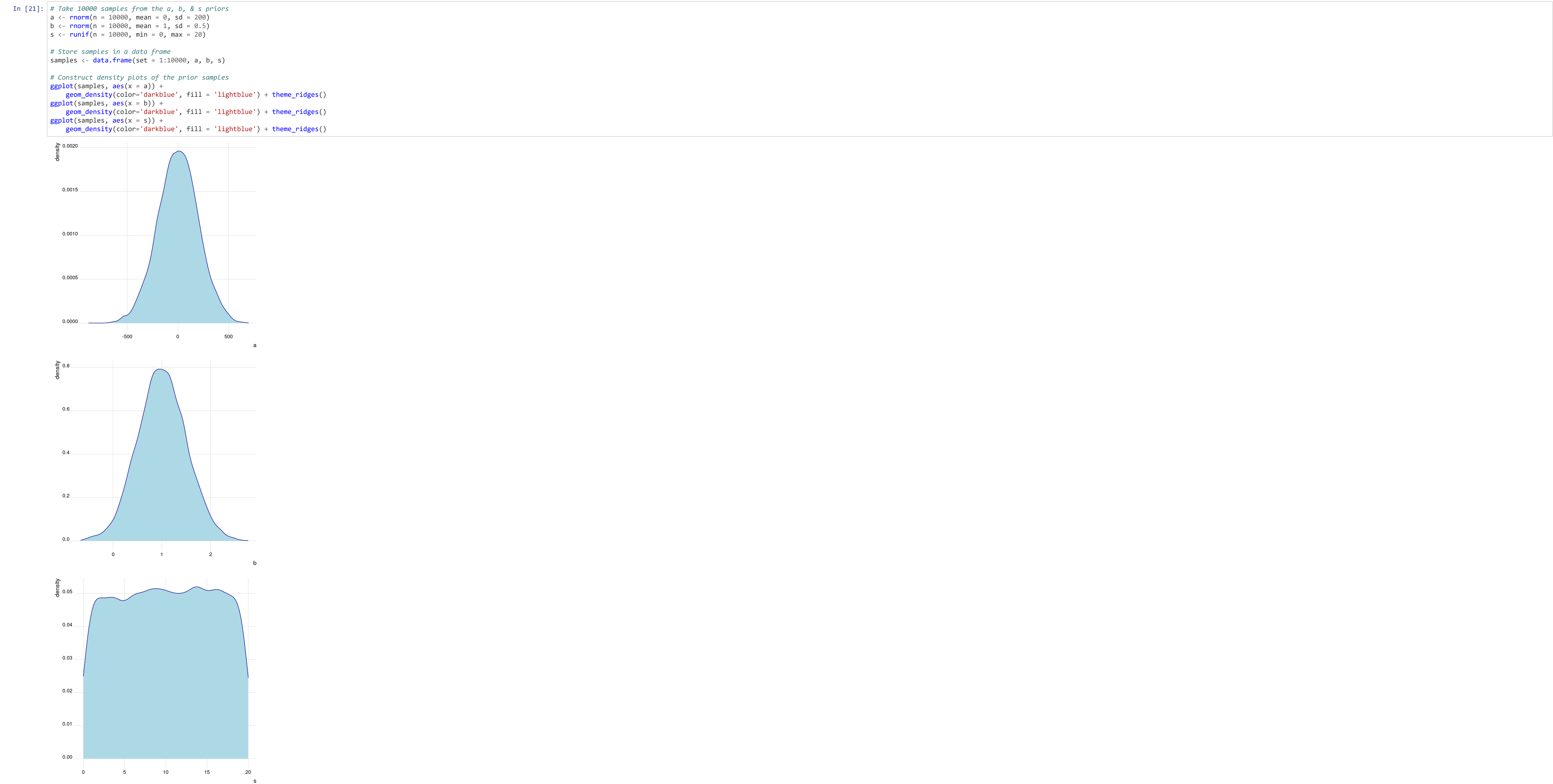
Mathematical notation

- $Y_i \sim N(m_i, s^2)$
- $m_i = a + bX_i$
- $a \sim N(0, 200^2)$
- $b \sim N(1, 0.5^2)$
- $s \sim Unif(0, 20)$

Regression priors

Let Y_i be the weight (in kg) of subject i . Past studies have shown that weight is linearly related to height X_i (in cm). The average weight m_i among adults of any shared height X_i can be written as $m_i = a + bX_i$. But height isn't a perfect predictor of weight - individuals vary from the trend. To this end, it's reasonable to assume that Y_i are Normally distributed around m_i with residual standard deviation s : $Y_i \sim N(m_i, s^2)$.

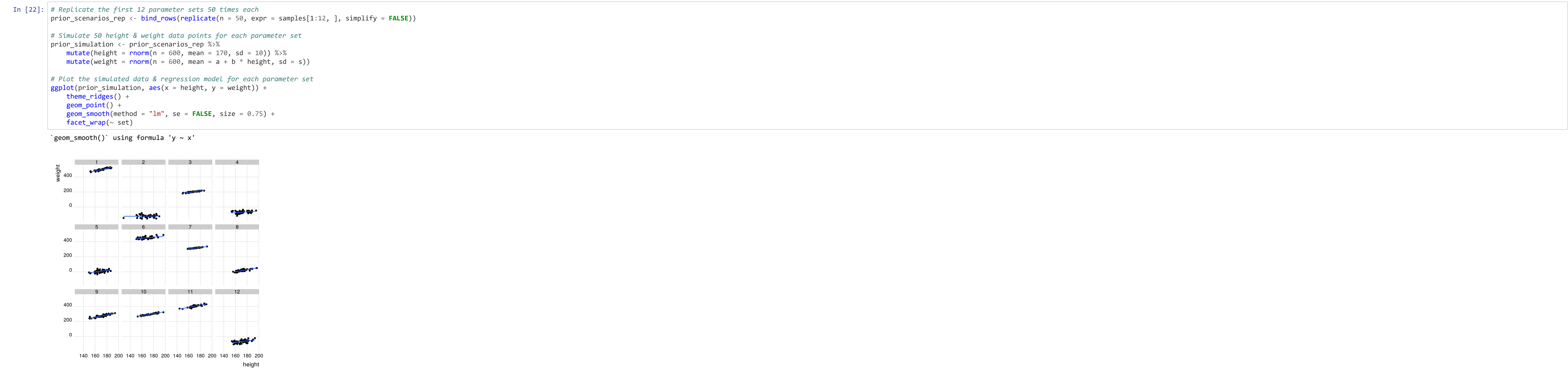
Note the 3 parameters in the model of weight by height: intercept a , slope b , & standard deviation s . In the first step of our Bayesian analysis, we will simulate the following prior models for these parameters: $a \sim N(0, 200^2)$, $b \sim N(1, 0.5^2)$, and $s \sim Unif(0, 20)$.



These simulations approximate our prior models of each separate model parameter. There's likely a positive association between weight & height ($b > 0$) but more uncertainty about the intercept a . Further, at any given height, the typical deviation of individuals' weights from the trend is equally likely to be anywhere between 0 and 20 kg.

Visualizing the regression priors

Previously, we simulated 10,000 samples for each parameter (a, b, s) in the Bayesian regression model of weight Y by height X : $Y \sim N(m, s^2)$ with mean $m = a + bX$. The set of a, b , and s values in each row of samples represents a prior plausible regression scenario. To explore the scope of these prior scenarios, we will simulate 50 pairs of height and weight values from each of the first 12 sets of prior parameters a, b , and s .



These 12 plots demonstrate the range of prior plausible models. These models have different intercepts, slopes, and residual standard deviations. Almost all of the models have positive slopes, demonstrating the prior information that there is likely a positive association between weight & height. Given our vague prior for a , some of these models are even biologically impossible.

Define, compile, & simulate the regression model

Upon observing the relationship between weight Y_i and height X_i for the 507 subjects i in the `bdims` data set, we can update our posterior model of this relationship. To build our posterior, we must combine our insights from the likelihood and priors:

- likelihood:** $Y_i \sim N(m_i, s^2)$ where $m_i = a + bX_i$
- priors:** $a \sim N(0, 2002)$, $b \sim N(1, 0.52)$ and $s \sim Unif(0, 20)$


```
In [23]: # DEFINE the model
weight_model <- "model{
  # Likelihood model for Y[i]
  for(i in 1:length(Y)) {
    Y[i] ~ dnorm(m[i], s^(-2))
    m[i] <- a + b * X[i]
  }
  # Prior models for a, b, s
  a ~ dnorm(0, 200^(-2))
  b ~ dnorm(1, 0.5^(-2))
  s ~ dunif(0, 20)
}"

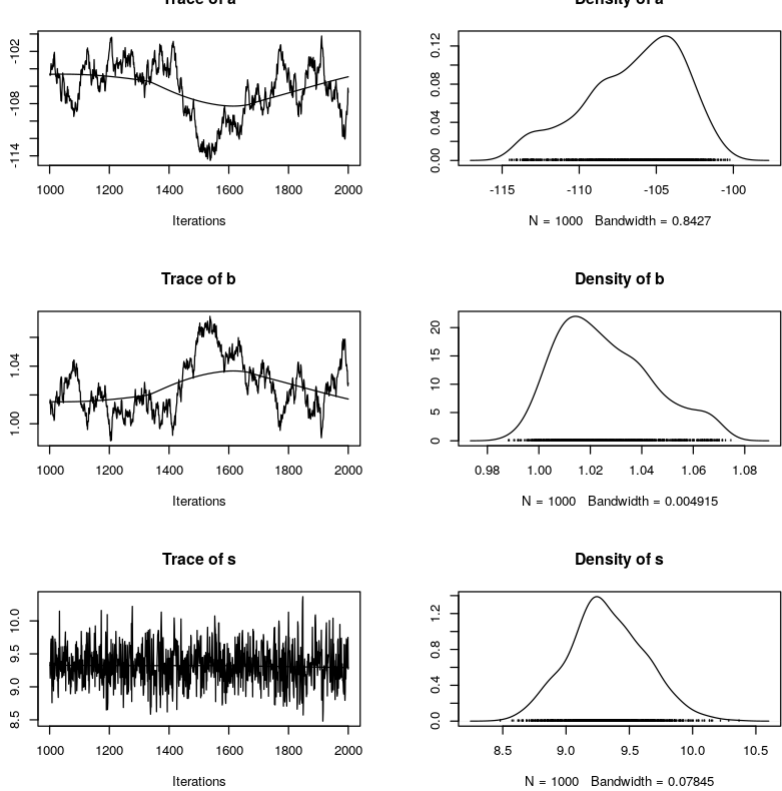
# COMPIL the model
weight_jags <- jags.model(textConnection(weight_model), data = list(Y = bdim$wtg, X = bdim$htg),
  inits = list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 1989))

# SIMULATE the posterior
weight_sim <- coda.samples(model = weight_jags, variable.names = c("a", "b", "s"), n.iter = 1000)

# PLOT the posterior
plot(weight_sim)

Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 507
Unobserved stochastic nodes: 3
Total graph size: 1321

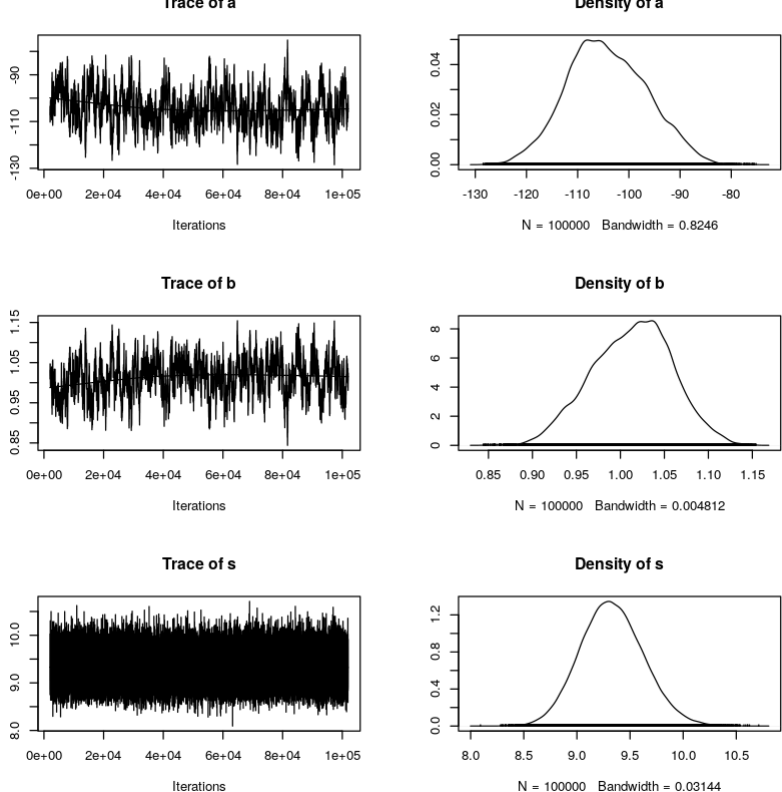
Initializing model
```



We've successfully defined, compiled, and simulated your Bayesian regression model. But the results don't look that great. Let's fix that.

```
In [24]: # SIMULATE the posterior 100,000 times
weight_sim <- coda.samples(model = weight_jags, variable.names = c("a", "b", "s"), n.iter = 100000)

# PLOT the posterior
plot(weight_sim)
```



Trace plots indicate that after only 1,000 iterations, the a and b parallel chains had not stabilized. However, after 100,000 iterations, the chains demonstrate greater stability. We might also increase the stability of our simulation by *standardizing* the height data.

Posterior estimation & inference

Posterior point estimates

Recall the likelihood of the Bayesian regression model of weight Y by height X : $Y \sim \mathcal{N}(m, s^2)$ where $m = a + bX$. A 100,000 iteration **RJAGS** simulation of the posterior.

The posterior means of the intercept & slope parameters, a & b , reflect the posterior mean trend in the relationship between weight & height. In contrast, the full posteriors of a & b reflect the range of plausible parameters, thus posterior uncertainty in the trend. We will examine the trend and uncertainty.

```
In [25]: # Summarize the posterior Markov chains
summary(weight_sim)

# Store the chains in a data frame
weight_chains <- data.frame(weight_sim[[1]])

# Calculate the estimated posterior mean of b
mean(weight_chains$b)

# Plot the posterior mean regression model
ggplot(bdim, aes(x = hgt, y = wgt)) +
  geom_point(alpha=0.5) +
  geom_abline(intercept = mean(weight_chains$a), slope = mean(weight_chains$b), color = "blue") +
  theme_ridges()

# Visualize the range of 20 posterior regression models
ggplot(bdim, aes(x = hgt, y = wgt)) +
  geom_point(alpha=0.5) +
  geom_abline(intercept = weight_chains$a[1:20], slope = weight_chains$b[1:20], color = "gray", size = 0.25) +
  theme_ridges()
```

```
Iterations = 2001:102000
Thinning interval = 1
Number of chains = 1
Sample size per chain = 1e+05

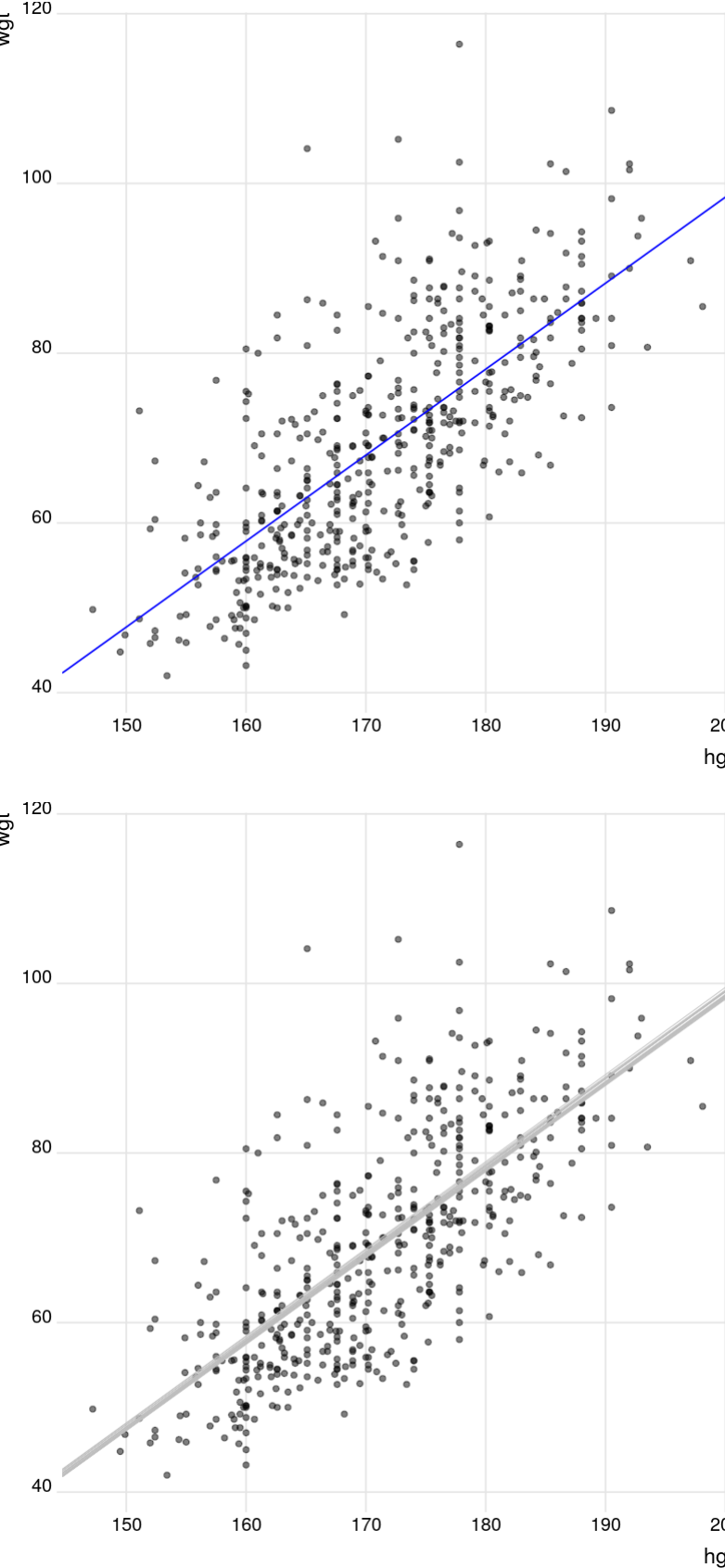
1. Empirical mean and standard deviation for each variable,
plus standard error of the mean:

      Mean      SD Naive SE Time-series SE
a -104.222  7.77968  0.0246015      0.646878
b   1.013  0.04539  0.0001435      0.003794
s    9.331  0.29656  0.0009378      0.001214

2. Quantiles for each variable:

      2.5%      25%      50%      75%      97.5%
a -118.8879 -109.7096 -104.681 -98.780 -88.712
b   0.9224   0.9813   1.016   1.045   1.098
s    8.7743   9.1273   9.323   9.526   9.938

1.01301864233895
```

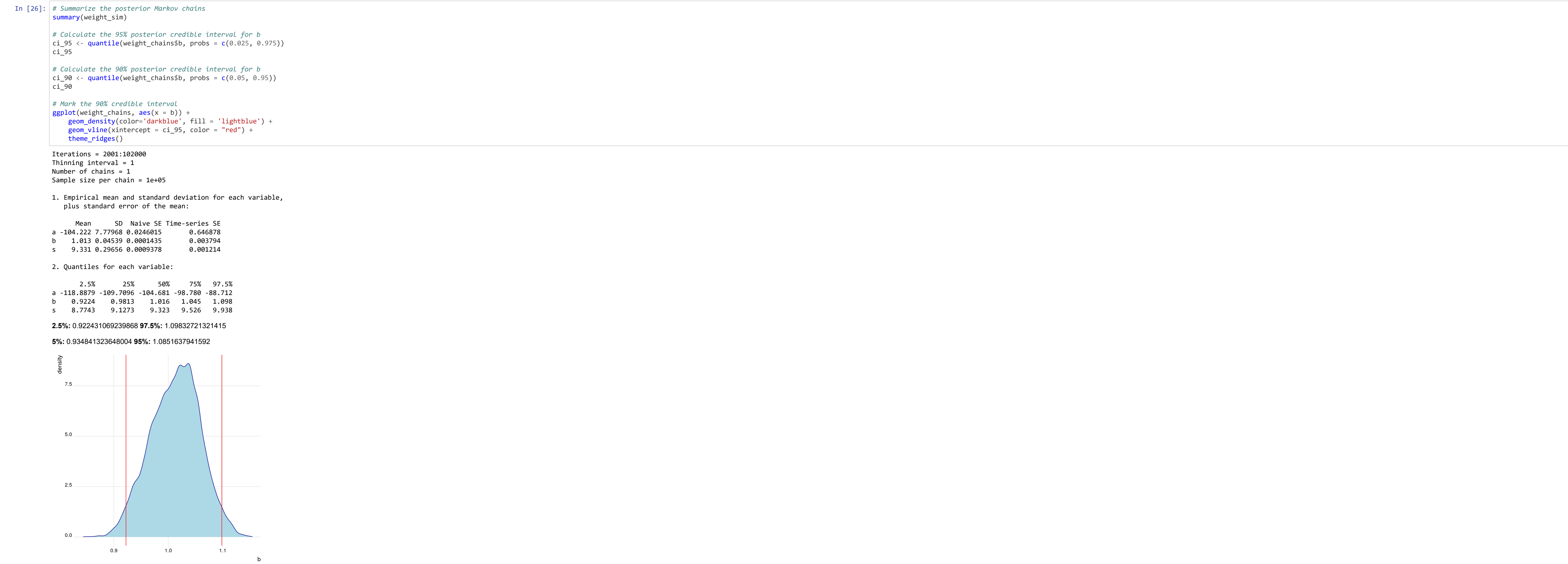


We now have a sense of the posterior mean trend in the linear relationship between weight and height as well as the posterior uncertainty in this trend. Given the size of the data and selection of priors, the posterior uncertainty is noticeably small as evidenced by the tight distribution of the gray posterior plausible lines around the trend.

Posterior credible intervals

Let's focus on slope parameter b , the rate of change in weight over height. The **posterior mean** of b reflects the trend in the posterior model of the slope. In contrast, a posterior **credible interval** provides a range of posterior plausible slope values, thus reflects posterior uncertainty about b . For example, the 95% credible interval for b ranges from the 2.5th to the 97.5th quantile of the b posterior. Thus there's a 95% (posterior) chance that b is in this range.

We will use **RJAGS** simulation output to approximate credible intervals for b . The 100,000 iteration **RJAGS** simulation of the posterior.



Based on our calculations we can say that there's a 95% (posterior) probability that, on average, the increase in weight per 1 cm increase in height is between 0.92 and 1.1 kg.

Posterior probabilities

We've used **RJAGS** output to explore and quantify the posterior trend & uncertainty b . We can also use **RJAGS** output to assess specific hypotheses. For example: What's the posterior probability that, on average, weight increases by more than 1.1 kg for every 1 cm increase in height? That is, what's the posterior probability that $b > 1.1$?

We will approximate this probability by the proportion of b Markov chain values that exceed 1.1.

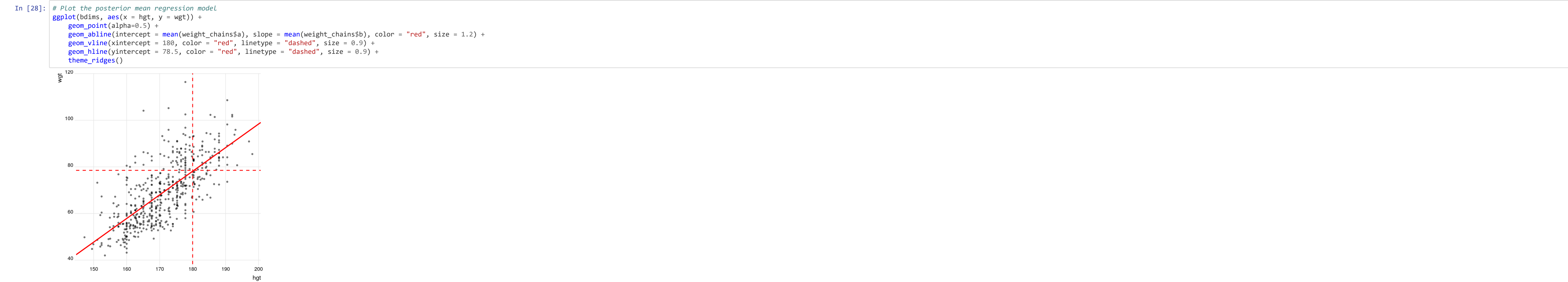


Based on our calculations we can say that there's only a $\sim 2\%$ (posterior) chance that, on average, the increase in weight per 1 cm increase in height exceeds 1.1 kg.

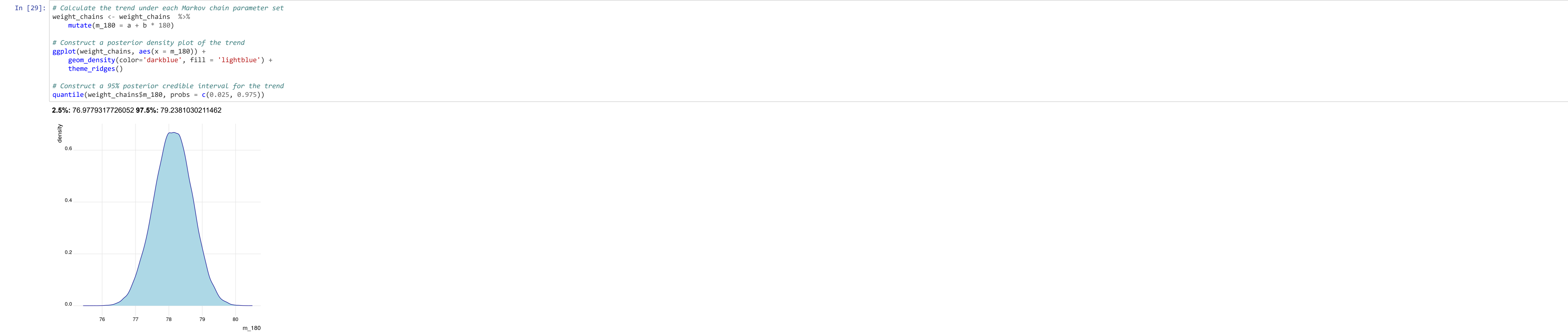
Posterior prediction

Inference for the posterior trend

Recall the likelihood of the Bayesian regression model of weight Y by height X : $Y \sim \mathcal{N}(m, s^2)$ where $m = a + bX$. Earlier we approximated the form of the posterior trend m (solid line). From this, notice that the typical weight among 180 cm adults is roughly 80 kg (dashed lines).



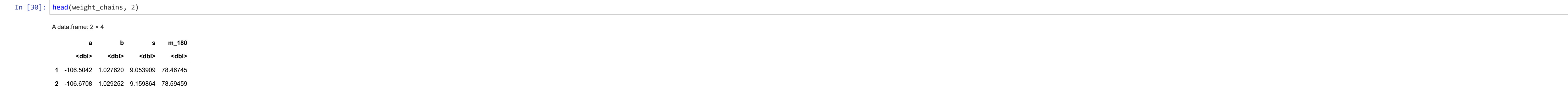
We will use **RJAGS** simulation output to approximate the posterior *trend* in weight among 180 cm tall adults as well as the posterior *uncertainty* in this trend.



The posterior trend of our regression model indicates that the typical weight among 180 cm tall adults is roughly 78 kg. However, posterior uncertainty in the regression model trickles down to uncertainty about this trend. This uncertainty is communicated through our credible interval: there's a 95% (posterior) chance that the typical weight at a height of 180 cm is between 76.98 and 79.24 kg.

Calculating posterior predictions

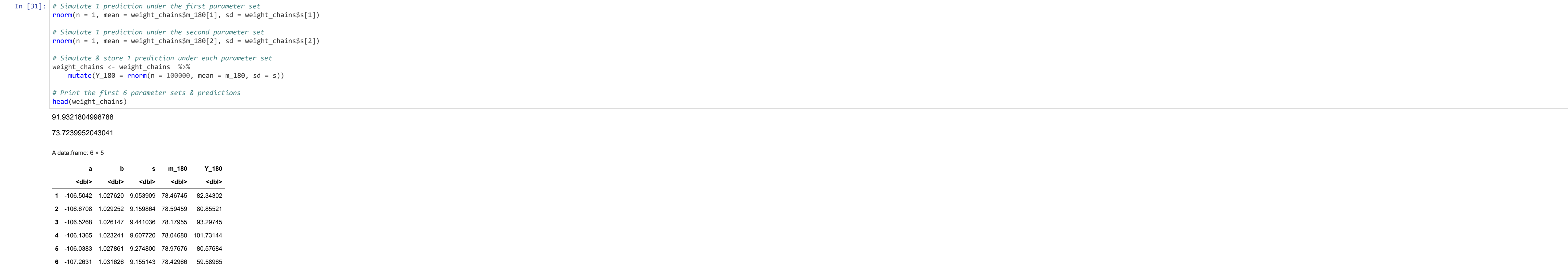
We just explored the posterior trend in weight Y among adults with height $X = 180$: $m_{180} = a + b \times 180$. The `weight_chains` data frame contains 100,000 posterior plausible values of m_{180} that we calculated from the corresponding values of a and b :



Forget the trend - what if we wanted to predict the weight of a specific 180 cm tall adult? We can! To do so, we must account for individual variability from the trend, modeled by:

$$Y_{180} \sim \mathcal{N}(m_{180}, s^2)$$

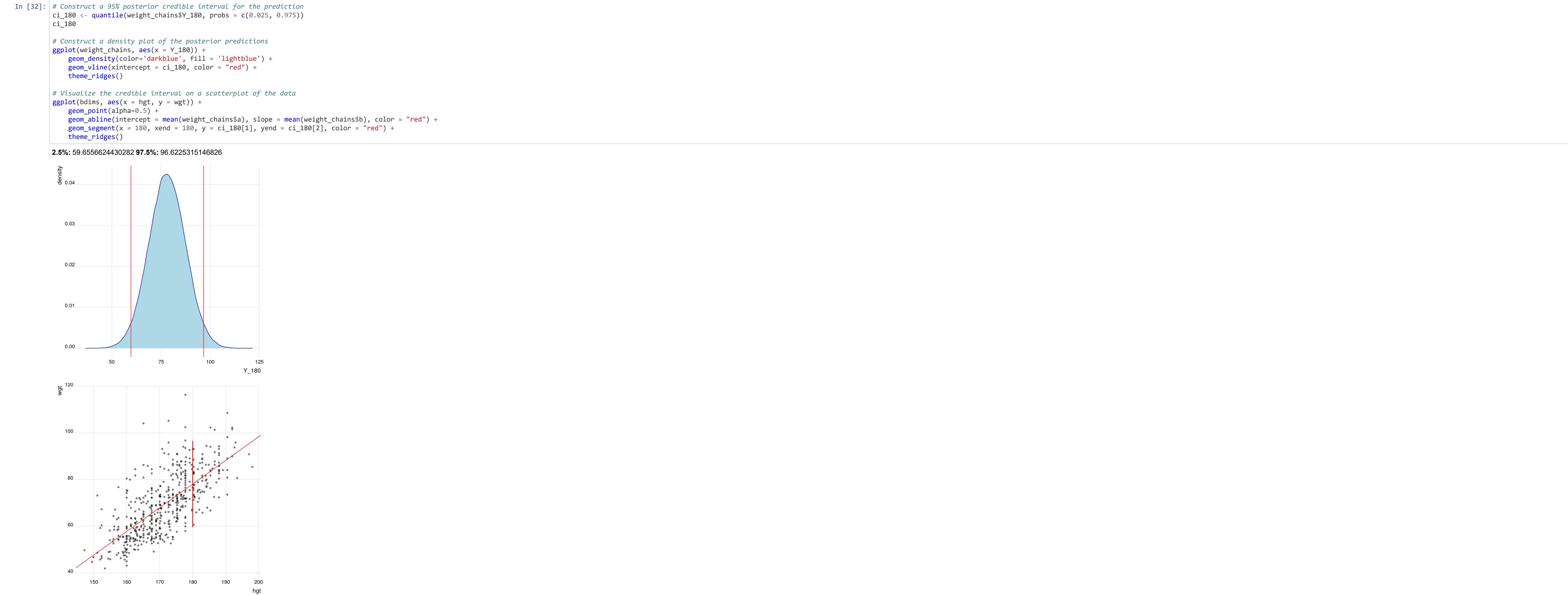
Using this model, we will simulate predictions of weight under each set of posterior plausible parameters in `weight_chains`.



We now have 100,000 predictions for the weight of a 180 cm tall adult that reflect the range of posterior plausible parameter settings.

Posterior predictive distribution

The `weight_chains` data frame contains our 100,000 posterior predictions, `Y_180`, for the weight of a 180 cm tall adult. We will use these 100,000 predictions to approximate the **posterior predictive distribution** for the weight of a 180 cm tall adult.



Bayesian regression with a categorical predictor

Modeling volume by weekday

- Y_i = trail volume (# of users) on day i
- X_i = 1 for weekdays, 0 for weekends

Model

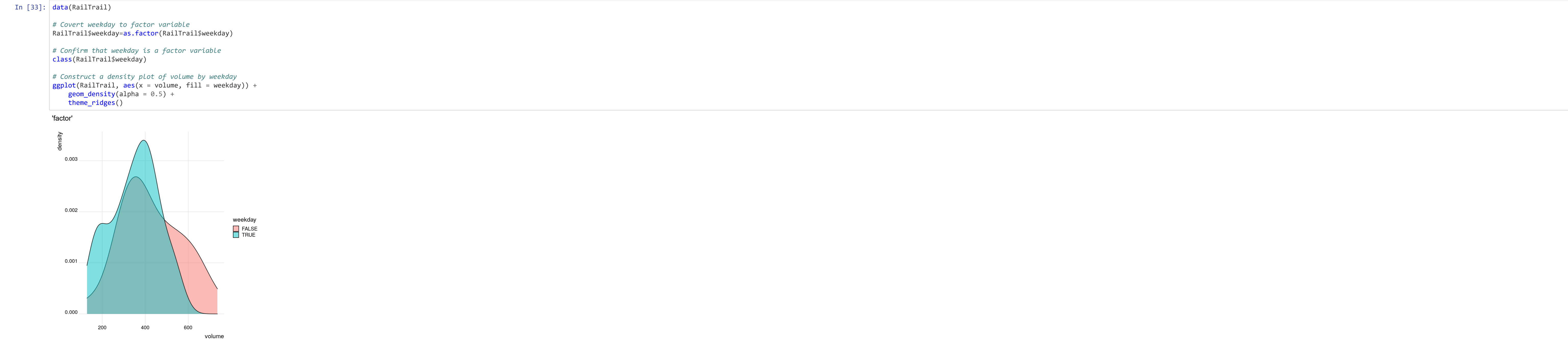
- $Y_i \sim N(m_i, s^2)$
- $m_i = a + bX_i$
- a = typical weekend volume
- $a + b$ = typical weekday volume
- b = contrast between typical weekday vs weekend volume
- s = residual standard deviation

Priors for a , b & s

- $Y_i \sim N(m_i, s^2)$
- $m_i = a + bX_i$
- a : Typical weekend volume is most likely around 400 users per day, but possibly as low as 100 or as high as 700 users.
 - $a \sim N(400, 100^2)$
- b : We lack certainty about how weekday volume compares to weekend volume. It could be more, it could be less.
 - $b \sim N(0, 200^2)$
- s : The standard deviation in volume from day to day (whether on weekdays or weekends) is equally likely to be anywhere between 0 and 200 users.
 - $s \sim Unif(0, 200)$

RailTrail sample data

The `RailTrail` data frame from the `mosaic` package contains data collected by the Pioneer Valley Planning Commission on the usage of a local rail-trail. For each of 90 days, they recorded the rail-trail volume (number of users) and whether it was a weekday (TRUE if yes and FALSE otherwise). We will explore the trends in weekday vs weekend volume.



Notice that, rail-trail volume tends to be slightly higher on weekends (~ 430 users per day) than on weekdays (~ 350 users per day).

RJAGS simulation with categorical variables

Consider the Normal regression model of volume Y_i by weekday status X_i :

- likelihood:** $Y_i \sim N(m_i, s^2)$ where $m_i = a + bX_i$
- priors:** $a \sim N(400, 100^2)$, $b \sim N(0, 200^2)$, $s \sim Unif(0, 200)$

We explored the relationship between Y_i and X_i for the 90 days recorded in `RailTrail`. In light of these data and the priors above, we will update our posterior model of this relationship. This differs from previous analyses in that X_i is categorical. In `rjags` syntax, its coefficient b is defined by two elements, `b[1]` and `b[2]`, which correspond to the weekend and weekday levels, respectively. For reference, `b[1]` is set to 0. In contrast, `b[2]` is modeled by the prior for b .


```
In [34]: # DEFINE the model
rail_model_1 <- "model{
  # Likelihood model for Y[i]
  for(i in 1:length(Y)){
    Y[i] ~ dnorm(m[i], s^(-2))
    m[i] <- a + b[X[i]]
  }

  # Prior models for a, b, s
  a ~ dnorm(400, 100^(-2))
  b[1] <- 0
  b[2] ~ dnorm(0, 200^(-2))
  s ~ dunif(0, 200)
}"

# COMPiLE the model
rail_jags_1 <- jags.model(textConnection(rail_model_1),
  data = list(Y = RailTrail$volume, X = RailTrail$weekday),
  inits = list(.RNG.name = "base:Wichmann-Hill", .RNG.seed = 10)
)

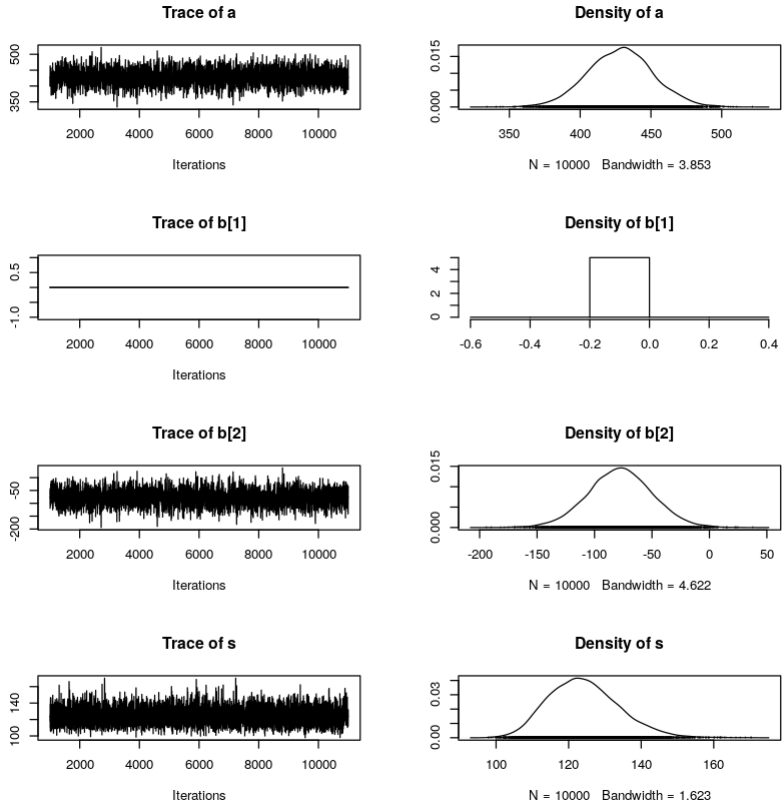
# SIMULATE the posterior
rail_sim_1 <- coda.samples(model = rail_jags_1, variable.names = c("a", "b", "s"), n.iter = 10000)

# Store the chains in a data frame
rail_chains_1 <- data.frame(rail_sim_1[[1]])

# PLOT the posterior
plot(rail_sim_1)
```

Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 90
Unobserved stochastic nodes: 3
Total graph size: 194

Initializing model



We've successfully defined, compiled, and simulated a Bayesian regression model with a categorical predictor.

Interpreting categorical coefficients

In our Bayesian model, $m_i = a + bX_i$ specified the dependence of typical trail volume on weekday status X_i (1 for weekdays and 0 for weekends). A `summary()` of your **RJAGS** model simulation provides posterior mean estimates of parameters a and b , the latter corresponding to $b.2$ here.

```
In [35]: summary(rail_sim_1)

Iterations = 1001:10000
Thinning interval = 1
Number of chains = 1
Sample size per chain = 10000

1. Empirical mean and standard deviation for each variable,
   plus standard error of the mean:

      Mean      SD Naive SE Time-series SE
a      428.47 23.092   0.23052       0.5312
b[1]      0.00  0.000   0.00000       0.0000
b[2]    -77.78 27.900   0.27900       0.6422
s      124.25  9.662   0.09662       0.1335

2. Quantiles for each variable:

      2.5%    25%    50%    75%   97.5%
a    383.1  412.9  428.78 443.62 474.20
b[1]   0.0    0.0    0.00   0.00   0.00
b[2]  -133.2  -96.3  -77.65  -59.43  -23.07
s     107.5  117.3  123.59  130.32  144.90
```

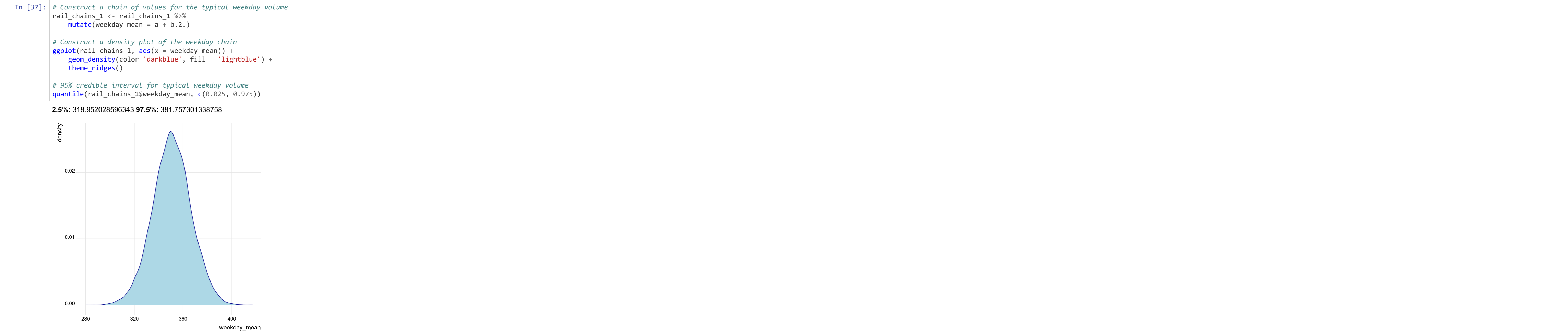
Typically, there are 428.47 trail users on a weekend day and 77.78 fewer users (~ 350.69) on a weekday.

Inference for volume by weekday

```
In [36]: head(rail_chains_1, 2)

A data frame: 2 x 4
      a      b.1.      b.2.      s
<dbl> <dbl> <dbl> <dbl>
1  420.6966    0 -54.30783 118.2328
2  399.5823    0 -52.02570 119.9499
```

These chains provide 10,000 unique sets of values for a , the typical trail volume on *weekend* days, and $b.2$, the **contrast** between typical weekday volume vs weekend volume. For example, the first set of parameters indicate that there are typically 420.6966 riders on weekend days and 54.30783 fewer riders on weekdays. Thus there are typically $420.6966 - 54.30783 = 366.3888$ riders on weekdays. We will utilize these simulation data to make inferences about weekday trail volume.



We've shown that there's a 95% posterior chance that the typical weekday volume is between 319 and 382 trail users.

Multivariate Bayesian regression

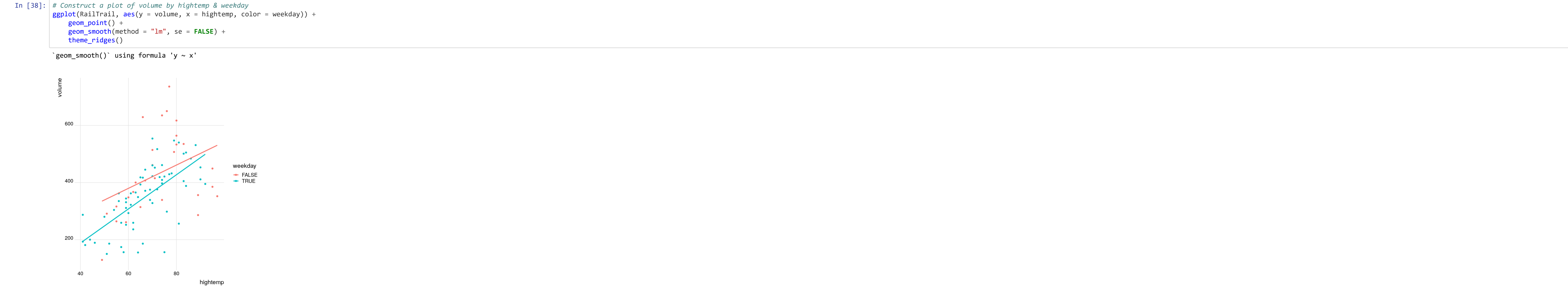
Modeling volume by temperature & weekday

- Y_i = trail volume (# of users) on day i
 - X_i = 1 for weekdays, 0 for weekends
 - Z_i = high temperature on day i (in °F)
 - $Y_i \sim N(m_i, s^2)$
 - $m_i = a + bX_i + cZ_i$
 - Weekends:** $m_i = a + cZ_i$
 - Weekdays:** $m_i = (a + b) + cZ_i$
 - a = weekend y-intercept
 - $a + b$ = weekday y-intercept
 - b = contrast between weekday vs weekend y-intercepts
 - c = common slope
 - s = residual standard deviation
- Priors for a, b, c & s
- $Y_i \sim N(m_i, s^2)$
 - $m_i = a + bX_i + cZ_i$
 - We lack certainty about the y-intercept for the relationship between temperature & *weekend* volume.
 - $a \sim N(0, 200^2)$
 - We lack certainty about how typical volume compares on weekdays vs weekends of similar temperature.
 - $b \sim N(0, 200^2)$
 - Whether on weekdays or weekends, we lack certainty about the association between trail volume & temperature.
 - $c \sim N(0, 20^2)$
 - The typical deviation from the trend is equally likely to be anywhere between 0 and 200 users.
 - $s \sim Unif(0, 200)$

Re-examining the RailTrail data

In our previous work, we observed that rail-trail volume tends to be lower on a weekday than a weekend. Some of the variability in volume might also be explained by outside temperature. For example, we might expect trail volume to increase on warm, pleasant days.

The `RailTrail` data set includes `hightemp`, the observed high temperature (°F) for each of the 90 days in the study period. We will use these data to explore the associations between trail volume, weekday status, and `hightemp`



Notice that for the 90 days in the study period, volume tends to increase with temperature. Further, volume tends to be higher on weekends than on weekdays of the same temperature.

RJAGS simulation for multivariate regression

Consider the following Bayesian model of volume Y_i by weekday status X_i and temperature Z_i :

- **likelihood:** $Y_i \sim N(m_i, s^2)$ where $m_i = a + bX_i + cZ_i$.
- **priors:** $a \sim N(0, 200^2)$, $b \sim N(0, 200^2)$, $c \sim N(0, 20^2)$, $s \sim Unif(0, 200)$

Our previous exploration of the relationship between volume, weekday, and hightemp in the RailTrail data provided some insight into this relationship. We will combine this with insight from the priors to develop a posterior model of this relationship using **RJAGS**.

```
In [39]: # DEFINE the model
rail_model_2 <- "model{
  # Likelihood model for Y[i]
  for(i in 1:length(Y)){
    Y[i] ~ dnorm(m[i], s^(-2))
    m[i] <- a + b[X[i]] + c * Z[i]
  }
  # Prior models for a, b, c, s
  a ~ dnorm(0, 200^(-2))
  b[1] <- 0
  b[2] ~ dnorm(0, 200^(-2))
  c ~ dnorm(0, 20^(-2))
  s ~ dunif(0, 200)
}"

# COMPIL the model
rail_jags_2 <- jags.model(textConnection(rail_model_2),
  data = list(Y = RailTrail$volume, X = RailTrail$weekday, Z = RailTrail$hightemp),
  inits = list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 10))

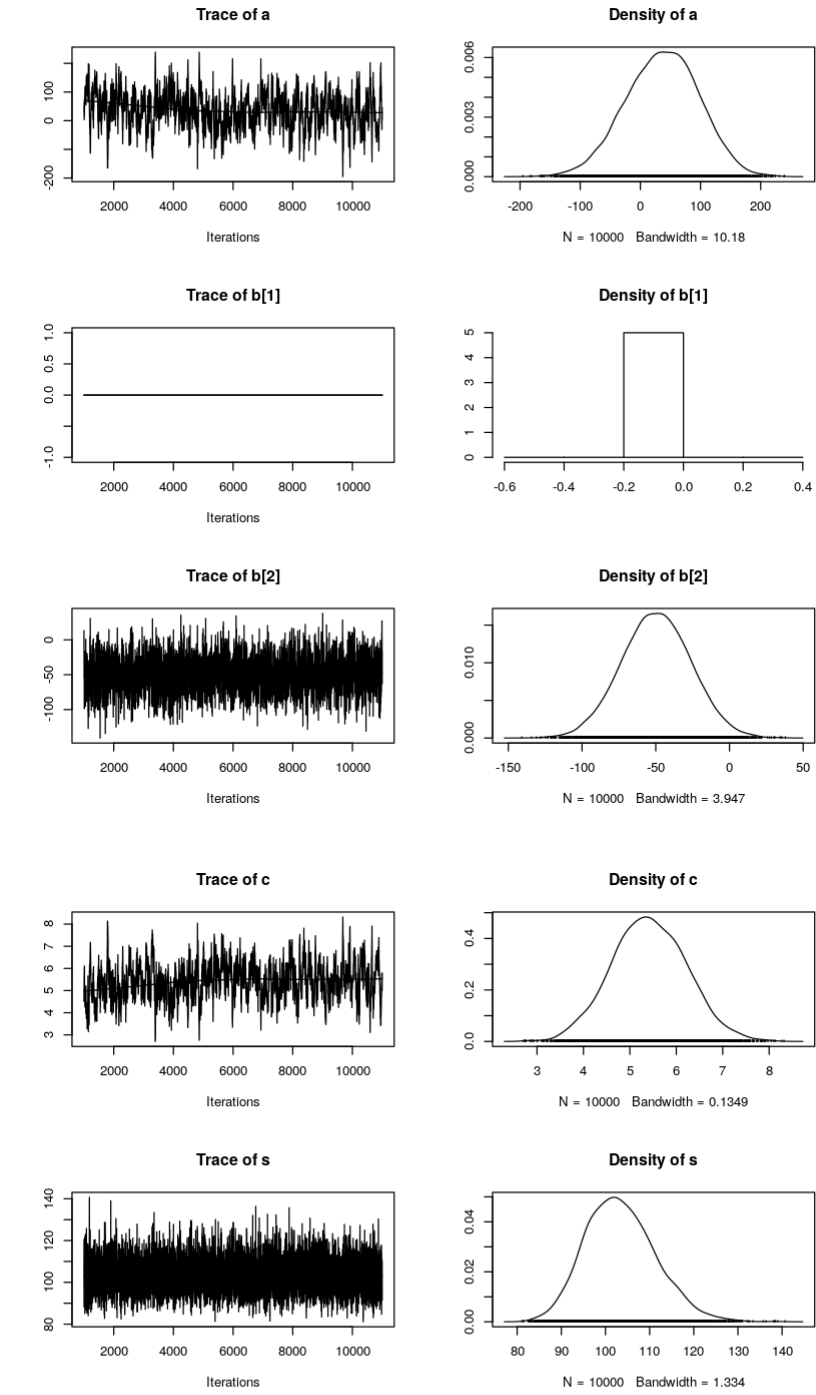
# SIMULATE the posterior
rail_sim_2 <- coda.samples(model = rail_jags_2, variable.names = c("a", "b", "c", "s"), n.iter = 10000)

# Store the chains in a data frame
rail_chains_2 <- data.frame(rail_sim_2[[1]])

# PLOT the posterior
plot(rail_sim_2)
```

Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 90
Unobserved stochastic nodes: 4
Total graph size: 385

Initializing model



Interpreting multivariate regression parameters

Our Bayesian model explored the dependence of typical trail volume on weekday status X_i and temperature Z_i : $m_i = a + bX_i + cZ_i$. A `summary()` of our **RJAGS** model simulation provides posterior mean estimates of parameters a , b , and c :

```
In [40]: summary(rail_sim_2)

Iterations = 1001:11000
Thinning interval = 1
Number of chains = 1
Sample size per chain = 10000

1. Empirical mean and standard deviation for each variable,
plus standard error of the mean:

      Mean      SD Naive SE Time-series SE
a    36.592  60.6238  0.606238      4.19442
b[1]    0.000  0.0000  0.000000      0.00000
b[2]  -49.610  23.4930  0.234930      0.55520
c      5.417   0.8029  0.080829      0.05849
s    103.434  7.9418  0.079418      0.11032

2. Quantiles for each variable:

      2.5%    25%    50%    75%   97.5%
a   -83.443  -4.631  38.160  78.413 150.306
b[1]    0.000  0.000  0.000  0.000  0.000
b[2]  -96.181 -65.468 -49.547 -33.633  -3.644
c      3.865   4.865   5.402   5.965   7.007
s     89.466  97.766 102.875 108.503 120.205
```

The posterior mean of c indicates that for both weekends and weekdays, typical rail volume increases by ~ 5.4 users for every 1 degree increase in temperature. The b coefficient represents typical volume is ~ 50 less on weekdays than on weekends of the same temperature.

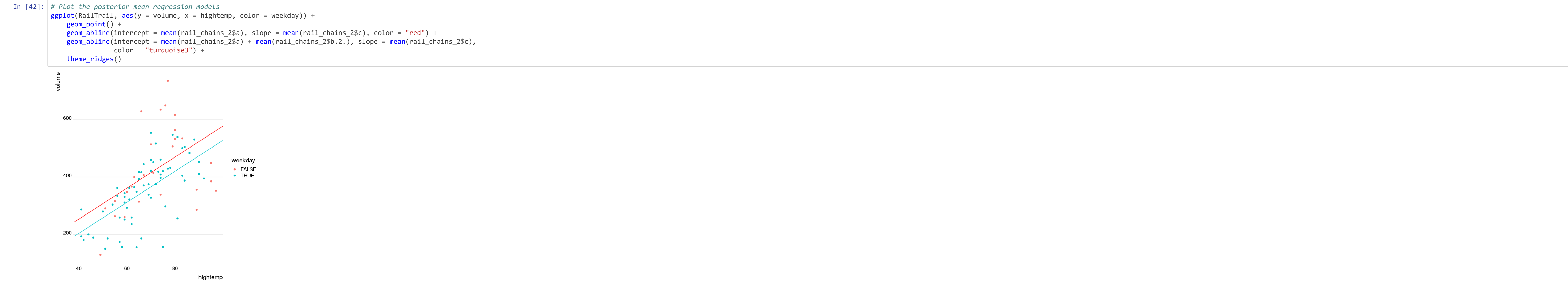
Posterior inference for multivariate regression

The 10,000 iteration **RJAGS** simulation output, `rail_sim_2` is a data frame of the Markov chain output:

```
In [41]: head(rail_chains_2, 2)

A data frame: 2 x 5
      a    b.1    b.2    c    s
<dbl> <dbl> <dbl> <dbl> <dbl>
1  40.76954    0 -12.6212 4.999202 111.02247
2  30.22211    0 -3.16221 4.853491  98.11892
```

We will use these 10,000 unique sets of parameter values to summarize the posterior mean trend in the relationships between trail volume, weekday status, and hightemp.



Our posterior analysis suggests that there's a positive association between volume and temperature. Further, the typical weekday volume is less than that on weekends of the same temperature.

Poisson regression

Normal likelihood structure

- Y = volume (# of users) on a given day
- $Y \sim N(m, s^2)$

Technically...

- The Normal model assumes Y has a continuous scale and can be negative.
- But Y is a discrete count and cannot be negative.

The Poisson model

- Y = volume (# of users) on a given day
- $Y \sim \text{Pois}(l)$
- Y is the # of independent events that occur in a fixed interval $(0, 1, 2, \dots)$.
- Rate parameter l represents the typical # of events per time interval ($l \geq 0$).

Poisson regression

- $Y_i \sim \text{Pois}(l_i)$ where $l_i > 0$
- $\log(l_i) = a + bX_i + cZ_i$

A solution:

- Linking l_i directly to the linear model assumes l_i can be negative, so we use a log link function to link l_i to the linear model. In turn:

$$l_i = e^{a+bX_i+cZ_i}$$

Mathematical Notation:

- $Y_i \sim \text{Pois}(l_i)$
- $\log(l_i) = a + bX_i + cZ_i$
- $a \sim N(0, 200^2)$
- $b \sim N(0, 2^2)$
- $c \sim N(0, 2^2)$

Caveats:

- $Y \sim \text{Pois}(l_i)$
- Assumption: Among days with similar temperatures and weekday status, variance in Y_i is equal to the mean of Y_i .
- Our data demonstrate potential **over-dispersion** - the variance is larger than the mean.
- Though not perfect, this model is an OK place to start.

RJAGS simulation for Poisson regression

Previously we engineered a Poisson regression model of volume Y_i by weekday status X_i and temperature Z_i :

- **likelihood:** $Y_i \sim \text{Pois}(l_i)$ where $\log(l_i) = a + bX_i + cZ_i$
- **priors:** $a \sim N(0, 200^2)$, $b \sim N(0, 2^2)$, and $c \sim N(0, 2^2)$

Combining our insights from the observed RailTrail data and the priors stated here, we will define, compile, and simulate a posterior model of this relationship using **RJAGS**.

```
In [43]: # DEFINE the model
poisson_model <- "model{
  # Likelihood model for Y[i]
  for(i in 1:length(Y)) {
    Y[i] ~ dpois(l[i])
    log(l[i]) <- a + b[X[i]] + c*Z[i]
  }
  # Prior models for a, b, c
  a ~ dnorm(0, 200^(-2))
  b[1] <- 0
  b[2] ~ dnorm(0, 2^(-2))
  c ~ dnorm(0, 2^(-2))
}"

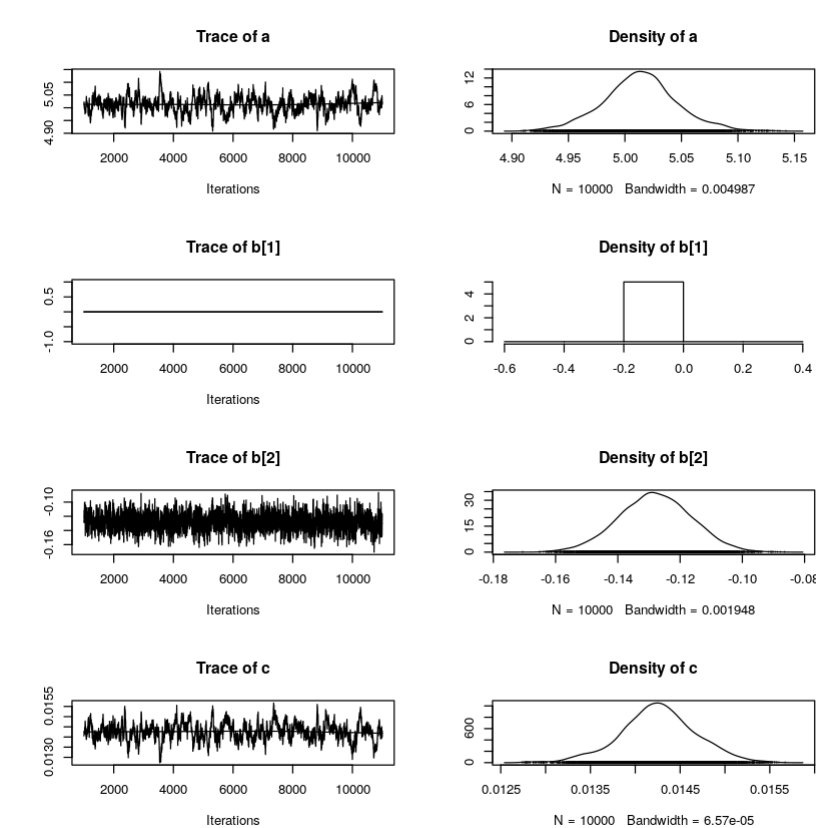
# COMPILE the model
poisson_jags <- jags.model(
  textConnection(poisson_model),
  data = list(Y = RailTrail$volume, X = RailTrail$weekday, Z = RailTrail$hightemp),
  inits = list(.RNG.name = "base::Wichmann-Hill", .RNG.seed = 10)
)

# SIMULATE the posterior
poisson_sim <- coda.samples(model = poisson_jags, variable.names = c("a", "b", "c"), n.iter = 10000)

# Store the chains in a data frame
poisson_chains <- data.frame(poisson_sim[[1]])

# PLOT the posterior
plot(poisson_sim)
```

Compiling model graph
Resolving undeclared variables
Allocating nodes
Graph information:
Observed stochastic nodes: 90
Unobserved stochastic nodes: 3
Total graph size: 441
Initializing model



Plotting the Poisson regression model

Recall the likelihood structure for our Bayesian Poisson regression model of volume Y_i by weekday status X_i and temperature Z_i : $Y_i \sim \text{Pois}(l_i)$ where,

- $\log(l_i) = a + bX_i + cZ_i$; thus
- $l_i = \exp(a + bX_i + cZ_i)$

Our 10,000 iteration **RJAGS** simulation of the model posterior, `poisson_sim`, is a data frame of the Markov chain output:

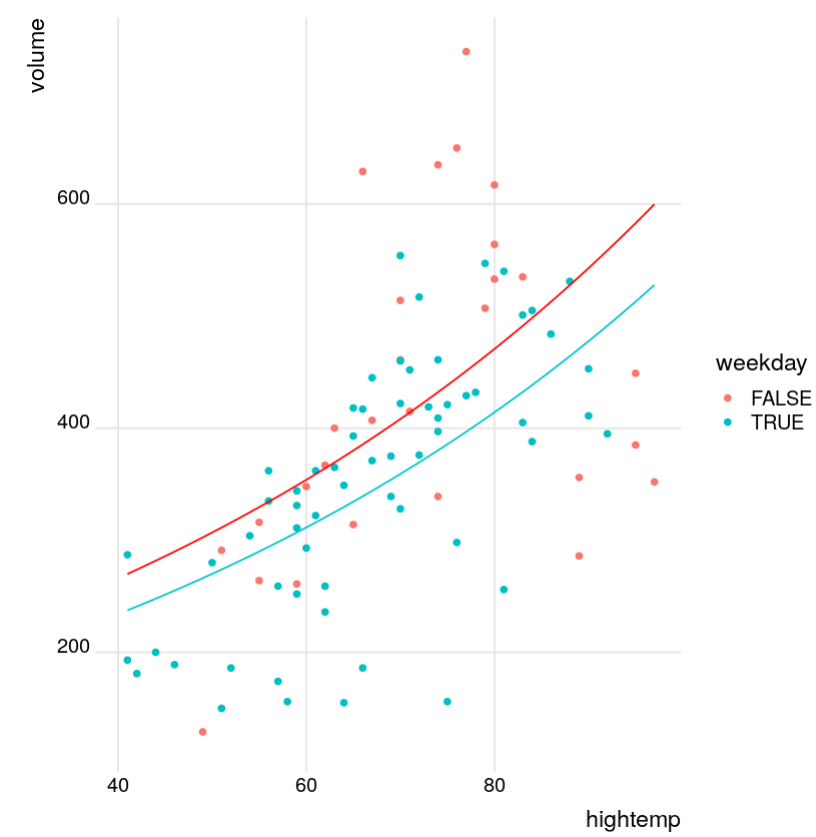
```
In [44]: head(poisson_chains, 2)
```

A data frame: 2 × 4

	a	b.1.	b.2.	c
	<dbl>	<dbl>	<dbl>	<dbl>
1	5.019807	0	-0.1222143	0.01405269
2	5.018642	0	-0.1217608	0.01407691

We will use these results to plot the posterior Poisson regression trends. These nonlinear trends can be added to a `ggplot()` using `stat_function()`.

```
In [45]: # Plot the posterior mean regression models
ggplot(RailTrail, aes(x = hightemp, y = volume, color = weekday)) +
  geom_point() +
  stat_function(fun = function(x){exp(mean(poisson_chains$a) + mean(poisson_chains$c) * x)}, color = "red") +
  stat_function(fun = function(x){exp(mean(poisson_chains$a) + mean(poisson_chains$b.2.) +
    mean(poisson_chains$c) * x)}, color = "turquoise") +
  theme_ridges()
```



Notice that unlike the Normal regression trend, the Poisson regression trend is curved.

Inference for the Poisson rate parameter

We will make inferences about the typical trail volume on 80 degree days.

```
In [46]: # Calculate the typical volume on 80 degree weekends & 80 degree weekdays
poisson_chains <- poisson_chains %>%
  mutate(l_weekend = exp(a + c * 80)) %>%
  mutate(l_weekday = exp(a + b.2. + c * 80))

# Construct a 95% CI for typical volume on 80 degree weekend
quantile(poisson_chains$l_weekend, c(0.025, 0.975))

# Construct a 95% CI for typical volume on 80 degree weekday
quantile(poisson_chains$l_weekday, c(0.025, 0.975))

2.5%: 462.143134922169 97.5%: 479.310055712039
2.5%: 407.464368507052 97.5%: 420.710174980761
```

Poisson posterior prediction

Our `l_weekday` variable reflects the *trend* in volume on 80 degree weekdays:

```
In [47]: head(poisson_chains, 2)
```

A data frame: 2 × 6

	a	b.1.	b.2.	c	l_weekend	l_weekday
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	5.019807	0	-0.1222143	0.01405269	465.9240	412.3235
2	5.018642	0	-0.1217608	0.01407691	466.2839	412.8292

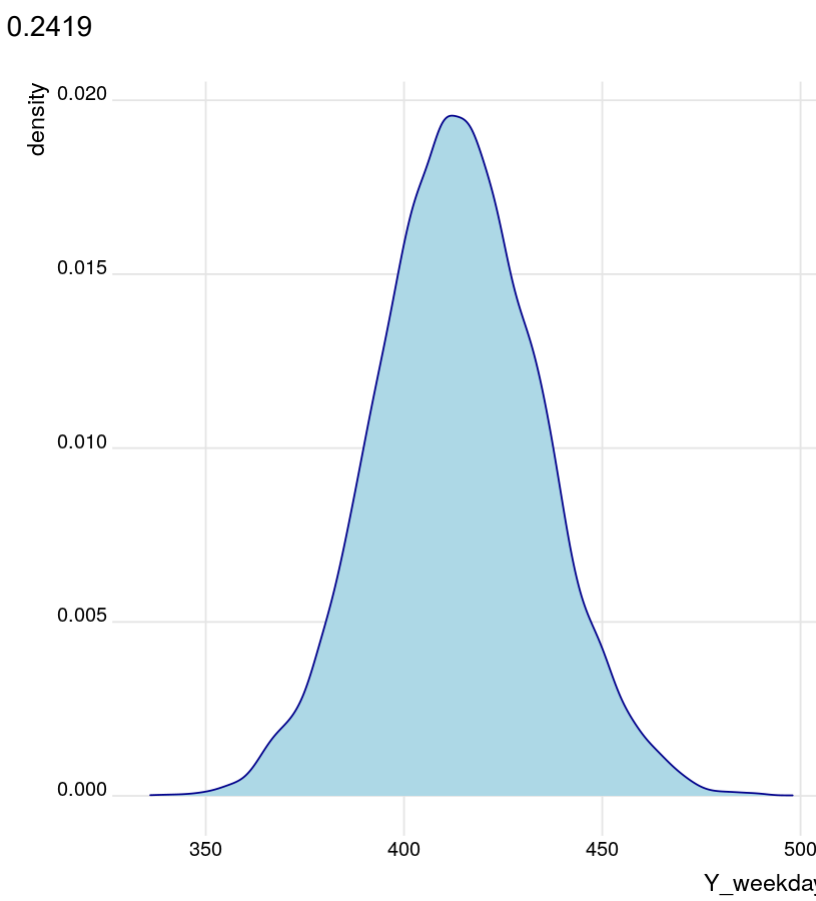
Now that we understand the *trend*, let's make some predictions. Specifically, let's *predict* trail volumes on the next 80 degree weekday. To do so, we must take into account individual variability from the *trend*, modeled by the likelihood $Y_i \sim \text{Pois}(l_i)$.

Using `rpois(n, lambda)` for sample size `n` and rate parameter `lambda`, we will simulate Poisson predictions of volume under each value of the posterior plausible trend in `poisson_chains`.

```
In [48]: # Simulate weekday predictions under each parameter set
poisson_chains <- poisson_chains %>%
  mutate(Y_weekday = rpois(n = 10000, lambda = l_weekday))

# Construct a density plot of the posterior weekday predictions
ggplot(poisson_chains, aes(x = Y_weekday)) +
  geom_density(color = "darkblue", fill = "lightblue") +
  theme_ridges()

# Posterior probability that weekday volume is less 400
mean(poisson_chains$Y_weekday < 400)
```



Recall that one of our motivations in applying the Poisson model was to accommodate the count nature of the volume data. This trickled down to our volume predictions `Y_weekday` - notice that these predictions, like the raw volume data, are discrete counts.

```
In [ ]:
```